

Supplementary Material for *Input-Adaptive Depth Routing in Pretrained Large Language Models with Mixture of Layers*

Lu Sun^{1,2} Rui Wen³ Jiayang Liu⁴ Jun Sakuma^{3,2}

¹Tohoku University ²RIKEN AIP ³Institute of Science Tokyo ⁴Nanyang Technological University

This document accompanies the main paper. It provides (i) the full experimental protocol (models, datasets, exact- L_{eff} matching, hyperparameters, seeding); (ii) a 3-seed variance analysis and the per-configuration expansion of the MoL row in Main Tables I and II; (iii) the sublayer granularity variant and (iv) the budget-exact MoL-TopK variant, the inference rule used for Main Fig. 6; (v) the full training procedure summarized in Main §III-D, together with (vi) the warm-start procedure and cold-start ablation and (vii) the KD recovery step (LoRA + KD), labelled “Stage B” here, with (viii) its design ablation (rows SK0–SK6, kept visibly distinct from the main-paper ablation in Main Table III); (ix) training cost and (x) engineering caveats; (xi) the complete proof of Main Theorem 1 via the Assumption 1 $\rightarrow$ Lemma 1 $\rightarrow$ Theorem 1 chain and the full proofs of Main Propositions 1–3; (xii) a computational-complexity analysis; and (xiii) an expanded related-work discussion. Section, equation, and citation numbers are local to this document; references to the main paper carry the prefix “Main”.

Contents

1	Expanded Related Work	2
1.1	Static Depth Pruning	2
1.2	Adaptive Depth and Dynamic Routing	2
1.3	Conditional Computation and Mixture-of-Experts	3
1.4	Comparison Matrix	3
2	Theoretical Analysis	3
2.1	Proof of Theorem 1 (Relative ℓ_2 Distance Shrinks with Depth)	4
2.2	Full Proofs of Main Propositions 1–3	5
2.3	Computational Complexity Analysis	7
3	The MoL Framework, Extended	8
3.1	MoL Variants	8
3.1.1	Sublayer Granularity: A Method Variant	8
3.1.2	MoL-TopK: A Budget-Exact Variant	10
3.2	Training Procedure	12
3.2.1	Relaxation, Temperature Curriculum, and Hyperparameters	12
3.2.2	Two-Stage Extension	14
3.2.3	Warm-Start from a Static Layer-Importance Prior	14
3.3	Stage B: Weight Recovery via Smart Knowledge Distillation	15
3.3.1	Goal and Overview	15
3.3.2	Two-Stage Training Protocol	16
3.3.3	Cost and Properties	17
3.3.4	Stage-B Recovery Design Ablation	17

4	Experimental Details	18
4.1	Detailed Experimental Setup	18
4.1.1	Backbones, Compression Levels, and Exact- L_{eff} Matching	18
4.1.2	Datasets, Metrics, Hyperparameters, and Seeds	19
4.1.3	Hardware, Code, and Reproducibility Recipe	20
4.2	Detailed Experimental Results	21
4.2.1	Per-Model 3-Seed Variance Analysis	22
4.2.2	Per-Variant Main Results: Layer vs. Sublayer, With vs. Without KD	23
4.2.3	FlexiDepth Pareto Comparison on Llama-3-8B-Instruct	24
4.3	Cost and Engineering Notes	26
4.3.1	Training Cost	26
4.3.2	Deployment and Serving Stack	26
4.3.3	Caveats and Engineering Notes	26

1 Expanded Related Work

For the main paper’s three-subsection related-work section, we provide an expanded discussion here that groups prior work by mechanism rather than by venue, and explicitly enumerates the comparison axes.

1.1 Static Depth Pruning

Static depth pruning removes whole layers or sublayer blocks using input-data statistics (BI, similarity, gradient) or greedy search: ShortGPT [1] (Block Influence ranking), FinerCut [2] (finer block decomposition), BlockPruner [3] (greedy PPL search over attention/FFN blocks, the strongest static baseline at aggressive compression in our experiments), LaCo [4] (merging adjacent layers rather than skipping), and 2SSP [5]/SLEB [6] (calibration-based selection at 30–40% removal). All commit to a single architecture applied uniformly across inputs; MoL makes the selection input-adaptive.

1.2 Adaptive Depth and Dynamic Routing

Adaptive-depth methods adjust the number or identity of layers per input. They split into early-exit, layer-skipping, and trinary-routing families.

Early exit. CALM [7], AdaInfer [8], and LITE [9] terminate inference at intermediate layers when confidence exceeds a threshold. These methods require auxiliary classifiers, careful calibration, and produce contiguous-prefix executions; they cannot drop interior layers while keeping later ones.

Layer skipping with independent gates. MoD [10] introduces a per-layer gate that decides whether to bypass the layer (or token within the layer). FFN-SkipLLM [11] and FLAP [12] apply similar gates at FFN granularity. Router-Tuning [13] trains an independent binary router per layer with a hinge objective and a frozen LLM. These methods all share the property that the decision at layer ℓ does not condition on decisions made at layers $1, \dots, \ell - 1$, except through the hidden state $\mathbf{x}_{\ell-1}$. MoL’s main paper argues, and the recurrence-removal ablation in Main Table III confirms, that a propagated recurrent state $\mathbf{z}_{\ell-1}$ adds meaningful structural information beyond what is recoverable from the hidden state alone.

Trinary routing with offline supervision. Dr.LLM [14] extends the gate to a trinary skip/execute/repeat decision and trains it via offline MCTS to generate the supervision target for each input. CoLa [15] performs online MCTS at inference. The MCTS pipelines are powerful but expensive: in Dr.LLM, generating the supervision targets dominates the overall cost. MoL preserves the trinary action space but replaces MCTS supervision with the warm-started budget hinge, dropping the expensive offline pipeline.

Recurrent and looped Transformers. Universal Transformer [16], Looped Transformer [17], latent-reasoning frameworks [18], and InnerThink [19] all use recurrent application of a fixed set of layers for iterative reasoning. These typically require training from scratch and a different architectural backbone; MoL’s *repeat* action achieves a closely related effect on a pretrained backbone without architectural change.

1.3 Conditional Computation and Mixture-of-Experts

Mixture-of-Experts (MoE) [20–22] activates a sparse subset of experts per token, scaling capacity along the *width* dimension, with extensions in adaptive expert routing [23–26] and PEFT-MoE hybrids [27–29]. MoL instead treats pretrained layers as reusable experts along *depth*; the two axes are orthogonal, and evaluating them jointly on MoE backbones is left to future work.

1.4 Comparison Matrix

Table 1 compares MoL against representative dynamic routing methods on the axes most relevant to deployment: input-adaptivity, retrofit cost, inference cost, training cost, parameter efficiency, router training cost, and whether the base LLM remains unchanged.

Table 1: Comparison of dynamic routing methods for frozen and partially-adapted LLMs. *Adaptive*: input-adaptive routing. *Retrofit*: added to pretrained LLM with minimal effort. *Cheap I*: efficient inference. *Cheap T*: efficient training. *Cheap P*: parameter-efficient. *Router T*: router trained end-to-end with LLM loss (✓) vs. via offline search-derived supervision (✗). *LLM[†]*: base LLM unchanged. ✓ = strong support, ✗ = not supported, – = not applicable.

Method	Adaptive	Retrofit	Cheap I	Cheap T	Cheap P	Router T	LLM [†]
CoLa [15]	✓	✓	✗	✗	✗	✗	✓
MoD [10]	✓	✗	✓	✗	✗	✓	✗
Universal Transformer [16]	✓	✗	✗	✗	✗	✓	✗
LLM-Pruner [30]	✗	✓	✓	✗	✗	–	✗
MoE [20]	✓	✗	✓	✗	✗	✓	✗
Mixture of Recursions [25]	✓	✗	✗	✗	✗	✓	✗
LayerSkip [31]	✗	✓	✓	✗	✗	–	✓
ShortGPT [1]	✗	✗	✓	✗	✗	–	✗
Router-Tuning [13]	✓	✓	✓	✗	✗	✓	✓
SkipGPT [32]	✓	✓	✓	✗	✗	✓	✓
FlexiDepth [33]	✓	✓	✗	✗	✗	✓	✓
Dr.LLM [14]	✓	✓	✓	✓	✗	✗	✓
MoL (Ours)	✓	✓	✓	✓	✓	✓	✓
MoL-TopK (Ours)	✓	✓	✓	✓	✓	✓	✓

Two rows need a note. SkipGPT [32] extends the layer-local gate family to per-token decisions, so its axis profile matches Router-Tuning at a small extra inference cost. MoL-TopK shares MoL’s profile but adds exact per-call budget compliance (§3.1.2), which the table does not show because every row above either enforces a budget on average or not at all.

2 Theoretical Analysis

This section collects the two formal analyses that support the framework: a self-contained proof of Theorem 1 (§2.1), and a complexity accounting of the routed forward pass (§2.3).

2.1 Proof of Theorem 1 (Relative ℓ_2 Distance Shrinks with Depth)

This subsection gives the full proof of Theorem 1 from the main paper. The structure matches Main Sec. III-C: Assumption 1 (bounded orthogonal increments) implies Lemma 1 (norm growth $\|\mathbf{x}_\ell\| = \Theta(\sqrt{\ell})$), and Theorem 1 (relative ℓ_2 decay) follows directly. We restate the setting for self-containedness.

Setting. Consider a pre-norm Transformer with hidden states $\{\mathbf{x}_\ell\}_{\ell=1}^L$ evolving as

$$\mathbf{x}_{\ell+1} = \mathbf{x}_\ell + f_\ell(\text{RMSNorm}(\mathbf{x}_\ell), \mathbf{W}_\ell),$$

where f_ℓ is the residual sublayer (self-attention or MLP) with parameters $\mathbf{W}_\ell$. RMSNorm is the variant used in LLaMA-style models [34, 35].

Assumption 1 (Bounded orthogonal increments; Main Assumption 1). (i) $\|f_\ell(\cdot)\| = \mathcal{O}(1)$ uniformly over ℓ . (ii) Residual increments are approximately orthogonal to the running stream: $\mathbb{E}\langle \mathbf{x}_\ell, f_\ell \rangle = o(\mathbb{E}\|\mathbf{x}_\ell\|^2)$. (iii) The layer field does not vanish with depth: $\mathbb{E}\|f_\ell\|^2 = \Omega(1)$ uniformly over ℓ .

Conditions (i) and (iii) together bracket the per-layer increment, $\mathbb{E}\|f_\ell\|^2 = \Theta(1)$, which is what makes the growth rate in Lemma 1 two-sided; (i) alone yields only the upper bound. Condition (iii) is stated as an explicit non-degeneracy requirement on the residual field rather than derived, and the results below are conditional on it holding for the stack under study.

Lemma 1 (Norm growth; Main Lemma 1). Under Assumption 1, $\mathbb{E}\|\mathbf{x}_\ell\|^2 = \Theta(\ell)$, hence $\|\mathbf{x}_\ell\| = \Theta(\sqrt{\ell})$. Equivalently, at initialization the expected squared norm of the hidden state at layer L satisfies the linear-in-depth bound

$$(1 + \frac{L}{2})d \leq \mathbb{E}[\|\mathbf{x}_{L,i}\|_2^2] \leq (1 + \frac{3L}{2})d,$$

recovering [36] as a special case.

We restate the theorem.

Theorem 1 (Relative ℓ_2 distance shrinks with depth; Main Theorem 1). Under Assumption 1 (equivalently, under Lemma 1),

$$\frac{\|\mathbf{x}_{\ell+1} - \mathbf{x}_\ell\|}{\|\mathbf{x}_\ell\|} = \mathcal{O}\left(\frac{1}{\sqrt{\ell}}\right).$$

Proof. We verify Assumption 1(i) holds for both attention and MLP sublayers, then apply Lemma 1 to bound the denominator, and combine.

(1) Bounded residual increment.

Attention sublayer. For multi-head self-attention with weights $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V, \mathbf{W}_O \in \mathbb{R}^{d \times d}$,

$$\begin{aligned} \mathbf{Q} &= \text{RMSNorm}(\mathbf{x}_\ell) \mathbf{W}_Q, \\ \mathbf{K} &= \text{RMSNorm}(\mathbf{x}_\ell) \mathbf{W}_K, \\ \mathbf{V} &= \text{RMSNorm}(\mathbf{x}_\ell) \mathbf{W}_V, \\ \mathbf{A} &= \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}}\right) \mathbf{V}, \\ f_\ell^{\text{attn}}(\mathbf{x}_\ell) &= \mathbf{A} \mathbf{W}_O. \end{aligned}$$

Since RMSNorm normalizes the root-mean-square to 1, $\|\text{RMSNorm}(\mathbf{x}_\ell)\| = \mathcal{O}(1)$. With Gaussian-initialized $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V$ having $\mathcal{O}(1)$ spectral norm, $\|\mathbf{Q}\|, \|\mathbf{K}\|, \|\mathbf{V}\| = \mathcal{O}(1)$. The softmax matrix is row-stochastic, so $\|\mathbf{A}\| = \mathcal{O}(1)$. Multiplying by $\mathbf{W}_O$ preserves the bound, giving $\|f_\ell^{\text{attn}}(\mathbf{x}_\ell)\| = \mathcal{O}(1)$.

MLP sublayer. For a two-layer MLP with pointwise nonlinearity σ and weights $\mathbf{W}_1 \in \mathbb{R}^{d \times d_{\text{ff}}}, \mathbf{W}_2 \in \mathbb{R}^{d_{\text{ff}} \times d}$,

$$\begin{aligned} \mathbf{u} &= \sigma(\text{RMSNorm}(\mathbf{x}_\ell) \mathbf{W}_1 + \mathbf{b}_1), \\ f_\ell^{\text{mlp}}(\mathbf{x}_\ell) &= \mathbf{W}_2 \mathbf{u} + \mathbf{b}_2. \end{aligned}$$

With $\|\sigma\|_\infty = \mathcal{O}(1)$ (true for GELU/SiLU/ReLU on bounded input) and $\mathcal{O}(1)$ spectral norms of $\mathbf{W}_1, \mathbf{W}_2$ at initialization, we obtain $\|f_\ell^{\text{mlp}}(\mathbf{x}_\ell)\| = \mathcal{O}(1)$.

(2) Hidden-state norm growth. The recursive update $\mathbf{x}_{\ell+1} = \mathbf{x}_\ell + f_\ell$ with bounded $\|f_\ell\|$ (Assumption 1(i)) and the orthogonality condition (Assumption 1(ii)) together let us drop the cross term $\langle \mathbf{x}_\ell, f_\ell \rangle$ in expectation: $\mathbb{E}\|\mathbf{x}_{\ell+1}\|^2 = \mathbb{E}\|\mathbf{x}_\ell\|^2 + \mathbb{E}\|f_\ell\|^2 + o(\mathbb{E}\|f_\ell\|^2)$. Conditions (i) and (iii) bound the increment on both sides, $c_1 \leq \mathbb{E}\|f_\ell\|^2 \leq c_2$ for constants $0 < c_1 \leq c_2$, so summing across layers gives $c_1 \ell(1 + o(1)) \leq \mathbb{E}\|\mathbf{x}_\ell\|^2 \leq c_2 \ell(1 + o(1))$, i.e. $\mathbb{E}\|\mathbf{x}_\ell\|^2 = \Theta(\ell)$ and $\|\mathbf{x}_\ell\| = \Theta(\sqrt{\ell})$, recovering Lemma 1. Dropping (iii) leaves the upper bound $\mathbb{E}\|\mathbf{x}_\ell\|^2 = \mathcal{O}(\ell)$, which suffices for a lower bound on the relative update but not for the rate below.

(3) Combining. The relative change is

$$\frac{\|\mathbf{x}_{\ell+1} - \mathbf{x}_\ell\|}{\|\mathbf{x}_\ell\|} = \frac{\|f_\ell\|}{\|\mathbf{x}_\ell\|} = \frac{\mathcal{O}(1)}{\Theta(\sqrt{\ell})} = \mathcal{O}\left(\frac{1}{\sqrt{\ell}}\right). \quad \square$$

Relation to prior work. The qualitative phenomenon — growing stream magnitude and diminishing relative deep-layer contribution in Pre-LN models — is documented empirically and theoretically in the “curse of depth” analysis [37]; Lemma 1 and the theorem give the rate in the form used by the skip prior of Main Eq. (8), namely a $\Theta(\sqrt{\ell})$ stream norm against a $\Theta(1)$ increment.

Interpretation. The theorem provides a structural reason to expect that ranking layers by an importance proxy (Block Influence in our case) yields a non-uniform prior over layer prunability. This is the justification for using BI specifically in the Stage-0 warm-start: at initialization, the depth-decay structure makes deep layers *a priori* more redundant, so the warm-start anchors the router in a structurally sensible region of the routing space rather than a uniform-skip region.

2.2 Full Proofs of Main Propositions 1–3

The main paper carries sketch proofs of Main Propositions 1–3 due to space; we give the detailed proofs here.

Proposition 1 (iteration-alignment criterion for repetition; full proof). Recall the statement. Fix a residual layer T_ℓ , query state $\mathbf{x}$, and target $\mathbf{x}^* = \mathbf{x} + \kappa f_\ell(\mathbf{x})$ with $\kappa > 1$. Let $J(\mathbf{x}') = \frac{1}{2}\|\mathbf{x}^* - \mathbf{x}'\|^2$ and define the iteration-alignment $\eta_\ell(\mathbf{x}) = \frac{\langle f_\ell(T_\ell(\mathbf{x})), f_\ell(\mathbf{x}) \rangle}{\|f_\ell(\mathbf{x})\|^2 + \varepsilon}$. We show

$$J(T_\ell^2(\mathbf{x})) - J(T_\ell(\mathbf{x})) = -(\kappa - 1)\|f_\ell(\mathbf{x})\|^2 \eta_\ell(\mathbf{x}) + \frac{1}{2}\|f_\ell(T_\ell(\mathbf{x}))\|^2. \quad (1)$$

Step 1: write the two iterates explicitly. Because $T_\ell(\mathbf{y}) = \mathbf{y} + f_\ell(\mathbf{y})$,

$$\begin{aligned} T_\ell(\mathbf{x}) &= \mathbf{x} + f_\ell(\mathbf{x}), \\ T_\ell^2(\mathbf{x}) &= T_\ell(\mathbf{x}) + f_\ell(T_\ell(\mathbf{x})). \end{aligned}$$

Hence $T_\ell^2(\mathbf{x}) - T_\ell(\mathbf{x}) = f_\ell(T_\ell(\mathbf{x}))$ and $\mathbf{x}^* - T_\ell(\mathbf{x}) = (\kappa - 1)f_\ell(\mathbf{x})$.

Step 2: expand the displacement objective. Let $\Delta := f_\ell(T_\ell(\mathbf{x}))$. Then

$$\begin{aligned} J(T_\ell^2(\mathbf{x})) &= \frac{1}{2}\|\mathbf{x}^* - T_\ell^2(\mathbf{x})\|^2 \\ &= \frac{1}{2}\|\mathbf{x}^* - T_\ell(\mathbf{x}) - \Delta\|^2 \\ &= \frac{1}{2}\|\mathbf{x}^* - T_\ell(\mathbf{x})\|^2 - \langle \mathbf{x}^* - T_\ell(\mathbf{x}), \Delta \rangle + \frac{1}{2}\|\Delta\|^2 \\ &= J(T_\ell(\mathbf{x})) - \langle \mathbf{x}^* - T_\ell(\mathbf{x}), \Delta \rangle + \frac{1}{2}\|\Delta\|^2. \end{aligned}$$

Subtracting $J(T_\ell(\mathbf{x}))$,

$$J(T_\ell^2(\mathbf{x})) - J(T_\ell(\mathbf{x})) = -\langle \mathbf{x}^* - T_\ell(\mathbf{x}), \Delta \rangle + \frac{1}{2}\|\Delta\|^2.$$

Step 3: substitute the alignment. Using $\mathbf{x}^* - T_\ell(\mathbf{x}) = (\kappa - 1) f_\ell(\mathbf{x})$ and $\Delta = f_\ell(T_\ell(\mathbf{x}))$,

$$\langle \mathbf{x}^* - T_\ell(\mathbf{x}), \Delta \rangle = (\kappa - 1) \langle f_\ell(\mathbf{x}), f_\ell(T_\ell(\mathbf{x})) \rangle = (\kappa - 1) \|f_\ell(\mathbf{x})\|^2 \eta_\ell(\mathbf{x}) + \mathcal{O}(\varepsilon),$$

where the $\mathcal{O}(\varepsilon)$ correction is the numerical-stability constant in the denominator of η_ℓ and vanishes in the formal statement. Combining the two displays gives (1).

Step 4: sign analysis. Setting $A := \|f_\ell(\mathbf{x})\|^2$ and $B := \|f_\ell(T_\ell(\mathbf{x}))\|^2$, (1) reads $-(\kappa - 1) A \eta_\ell(\mathbf{x}) + \frac{1}{2} B$. So repetition *strictly reduces* J iff

$$(\kappa - 1) A \eta_\ell(\mathbf{x}) > \frac{1}{2} B.$$

Under the field-magnitude-preserving regime $B \approx A$ (an empirically supported approximation when T_ℓ is mildly contractive locally), this simplifies to $\eta_\ell(\mathbf{x}) > \frac{1}{2(\kappa-1)}$. Conversely $\eta_\ell(\mathbf{x}) \leq 0$ makes the leading term non-negative while $\frac{1}{2} B \geq 0$, so the difference is non-negative — repetition strictly increases (or leaves unchanged) J . For $\kappa \gg 1$ the threshold $\frac{1}{2(\kappa-1)} \rightarrow 0$ and the criterion reduces to $\eta_\ell(\mathbf{x}) > 0$, which is what Main Proposition 1 states. $\square$

Corollary (Banach contraction case). If T_ℓ is a strict contraction in a neighbourhood of a fixed point $\mathbf{x}^\circ$, i.e. $\|T_\ell(\mathbf{y}) - \mathbf{x}^\circ\| \leq \gamma \|\mathbf{y} - \mathbf{x}^\circ\|$ for some $\gamma < 1$ and all $\mathbf{y}$ near $\mathbf{x}^\circ$, then $\eta_\ell(\mathbf{y}) > 0$ throughout the basin and iterating T_ℓ realises fixed-point iteration with geometric convergence to $\mathbf{x}^\circ$. This is the deep-equilibrium / looped-Transformer interpretation referenced after Main Proposition 1.

Proposition 2 (suboptimality of input-independent schedules; full proof). Recall the statement. Let $\mathbf{u}$ denote the input random variable on the data distribution $\mathcal{D}$, let $c^*(\mathbf{u})$ be any selected minimizer of the conditional loss–compute curve $c \mapsto \mathcal{L}_{\text{LM}}(\mathbf{u} \mid c)$ (the arg min may be set-valued; fix one measurable selection), and assume the selection varies across inputs, $\text{Var}_{\mathbf{u}}[c^*(\mathbf{u})] > 0$. We show that any *feasible* input-independent policy π — one assigning the same realised compute c to every input and satisfying $\mathbb{E}_{\mathbf{u}}[c(\mathbf{m})] \leq c^*$ — is strictly dominated by an input-adaptive perturbation at the same expected compute.

Step 1: feasible constant policies. A constant policy assigns the same realised compute c to every input, and feasibility under the inequality budget requires $c \leq c^*$. Since $\mathcal{L}_{\text{LM}}$ is monotone non-increasing in available compute, the best constant policy saturates the budget, so it suffices to treat $c = c^*$; the argument below applies verbatim at any $c < c^*$ with c in place of c^* .

Step 2: partition the input set by sign of $c^(\mathbf{u}) - c^*$.* Define

$$\mathcal{U}_- := \{\mathbf{u} : c^*(\mathbf{u}) < c^*\}, \quad \mathcal{U}_+ := \{\mathbf{u} : c^*(\mathbf{u}) > c^*\}.$$

By the variance hypothesis, both $\Pr_{\mathbf{u}}[\mathcal{U}_-]$ and $\Pr_{\mathbf{u}}[\mathcal{U}_+]$ are strictly positive (otherwise the cdf of $c^*(\mathbf{u})$ would degenerate to a point mass and the variance would vanish).

Step 3: construct a budget-preserving perturbation. Choose any $\delta_- > 0$ small enough that $c^* - \delta_-$ is still above each $c^*(\mathbf{u})$ for $\mathbf{u} \in \mathcal{U}_-$ in a positive-measure subset $\mathcal{U}_-^\delta \subseteq \mathcal{U}_-$, and analogously $\delta_+ > 0$ such that $c^* + \delta_+$ is still below $c^*(\mathbf{u})$ for $\mathbf{u}$ in some positive-measure subset $\mathcal{U}_+^\delta \subseteq \mathcal{U}_+$. Define the perturbed policy

$$\pi'(\mathbf{u}) := \begin{cases} c^* - \delta_-, & \mathbf{u} \in \mathcal{U}_-^\delta, \\ c^* + \delta_+, & \mathbf{u} \in \mathcal{U}_+^\delta, \\ c^*, & \text{otherwise.} \end{cases}$$

Choose (δ_-, δ_+) so that $\Pr_{\mathbf{u}}[\mathcal{U}_-^\delta] \delta_- = \Pr_{\mathbf{u}}[\mathcal{U}_+^\delta] \delta_+$; this preserves $\mathbb{E}_{\mathbf{u}}[c(\mathbf{m})] = c^*$, so the perturbed policy is feasible.

Step 4: strict loss decrease. Because $\mathcal{L}_{\text{LM}}(\mathbf{u} \mid c)$ is locally smooth and minimised at $c^*(\mathbf{u})$ for each $\mathbf{u}$, a first-order Taylor expansion gives, for the over-allocated subset $\mathcal{U}_-^\delta$,

$$\mathcal{L}_{\text{LM}}(\mathbf{u} \mid c^* - \delta_-) - \mathcal{L}_{\text{LM}}(\mathbf{u} \mid c^*) = -\delta_- \partial_c \mathcal{L}_{\text{LM}}(\mathbf{u} \mid c^*) + o(\delta_-).$$

For $\mathbf{u} \in \mathcal{U}_-^\delta$ we have $c^* > c^*(\mathbf{u})$, so by the Bayes-optimum definition the partial derivative $\partial_c \mathcal{L}_{\text{LM}}(\mathbf{u} \mid c^*)$ is *strictly positive*. Hence the bracketed difference is strictly negative for small enough δ_- . The same argument with the sign flipped shows that on $\mathcal{U}_+^\delta$ the difference $\mathcal{L}_{\text{LM}}(\mathbf{u} \mid c^* + \delta_+) - \mathcal{L}_{\text{LM}}(\mathbf{u} \mid c^*)$ is also strictly negative.

Step 5: aggregate. Taking expectations over $\mathbf{u}$ and integrating only over $\mathcal{U}_-^\delta \cup \mathcal{U}_+^\delta$ (where the change is strictly negative),

$$\mathbb{E}_{\mathbf{u}}[\mathcal{L}_{\text{LM}}(\mathbf{u} \mid \pi'(\mathbf{u}))] < \mathbb{E}_{\mathbf{u}}[\mathcal{L}_{\text{LM}}(\mathbf{u} \mid c^*)].$$

Therefore the constant policy $c \equiv c^*$ is not a local minimum of the master problem, and the variance hypothesis guarantees that such a perturbation exists. Every feasible input-independent policy is thus strictly dominated by an input-adaptive policy at the same expected compute. $\square$

Remark (KKT water-filling). The first-order condition for the master problem with a budget multiplier $\lambda \geq 0$ is $\partial_c \mathcal{L}_{\text{LM}}(\mathbf{u} \mid c(\mathbf{u})) + \lambda = 0$ for $\mathbf{u}$ in the support, i.e. the marginal LM-loss reduction per unit compute is equalised across inputs at the optimum. This is the classical water-filling characterisation referenced in the main text after Main Proposition 2; the recurrent state $\mathbf{z}_\ell$ of MoL approximates this water-filling solution by accumulating running compute as the policy is unrolled along the depth.

Proposition 3 (strict generalization; full proof). Recall the statement and the function classes. For a frozen L -layer stack with layer maps $T_1, \dots, T_L$,

$$\begin{aligned} \mathcal{F}_{\text{skip}}^L &:= \{ T_{i_K} \circ \dots \circ T_{i_1} : \{i_1 < \dots < i_K\} \subseteq [L] \}, \\ \mathcal{F}_{\text{rep}}^L &:= \{ T_L^{(n_L)} \circ \dots \circ T_1^{(n_1)} : n_\ell \in \{1, 2\} \}, \\ \mathcal{F}_{\text{MoL}}^L &:= \{ T_L^{(m_L)} \circ \dots \circ T_1^{(m_1)} : m_\ell \in \{0, 1, 2\} \}. \end{aligned}$$

We show $\mathcal{F}_{\text{skip}}^L \cup \mathcal{F}_{\text{rep}}^L \subsetneq \mathcal{F}_{\text{MoL}}^L$ for all $L \geq 3$.

Step 1: inclusion. $\mathcal{F}_{\text{skip}}^L$: a skip-only static schedule with executed indices $\{i_1, \dots, i_K\}$ corresponds to the MoL schedule $\mathbf{m}$ with $m_\ell = 1$ for $\ell \in \{i_1, \dots, i_K\}$ and $m_\ell = 0$ otherwise; this lies in $\mathcal{F}_{\text{MoL}}^L$. $\mathcal{F}_{\text{rep}}^L$: a repeat-only schedule with $n_\ell \in \{1, 2\}$ is the MoL schedule $\mathbf{m}$ with $m_\ell = n_\ell$, also in $\mathcal{F}_{\text{MoL}}^L$. Hence $\mathcal{F}_{\text{skip}}^L \cup \mathcal{F}_{\text{rep}}^L \subseteq \mathcal{F}_{\text{MoL}}^L$.

Step 2: strict containment. Consider the MoL schedule $\mathbf{m}^\dagger := (0, 2, 1)$ on layers $(1, 2, 3)$, corresponding to the composition

$$\mathcal{T}^\dagger := T_3 \circ T_2 \circ T_2,$$

which lies in $\mathcal{F}_{\text{MoL}}^3$. We claim $\mathcal{T}^\dagger \notin \mathcal{F}_{\text{skip}}^3 \cup \mathcal{F}_{\text{rep}}^3$.

Step 3a: $\mathcal{T}^\dagger \notin \mathcal{F}_{\text{skip}}^3$. Every element of $\mathcal{F}_{\text{skip}}^3$ is a composition of distinct T_ℓ 's, each applied exactly once. The schedule $(0, 2, 1)$ applies T_2 twice, so the only way to write it as an $\mathcal{F}_{\text{skip}}^3$ element would be to invoke a repeated factor — not allowed by the definition.

Step 3b: $\mathcal{T}^\dagger \notin \mathcal{F}_{\text{rep}}^3$. Every element of $\mathcal{F}_{\text{rep}}^3$ has all $n_\ell \in \{1, 2\}$, i.e. $n_\ell \neq 0$, so every layer fires at least once. The schedule $(0, 2, 1)$ has $m_1 = 0$, so T_1 does not fire — not allowed by the definition.

Step 4: extension to general $L \geq 3$. For any $L \geq 3$, embed the schedule $(0, 2, 1, 1, 1, \dots, 1)$ (skip then repeat then exec on layers 1, 2, 3; exec on layers 4, $\dots$, L). The same argument shows this composition is in $\mathcal{F}_{\text{MoL}}^L$ but in neither $\mathcal{F}_{\text{skip}}^L$ (because T_2 is repeated) nor $\mathcal{F}_{\text{rep}}^L$ (because T_1 is skipped). Hence strict containment holds for every $L \geq 3$. $\square$

Counts. A schedule has $|\mathcal{F}_{\text{skip}}^L| = 2^L$ (each layer kept or dropped), $|\mathcal{F}_{\text{rep}}^L| = 2^L$ (each layer applied once or twice), and $|\mathcal{F}_{\text{MoL}}^L| = 3^L$. For $L = 32$ (Llama-2-7B), $|\mathcal{F}_{\text{MoL}}^{32}|/|\mathcal{F}_{\text{skip}}^{32}| = (3/2)^{32} \approx 1.13 \times 10^5$, so MoL searches a schedule space $\sim 10^5 \times$ larger than skip-only static pruning on the same backbone, and strictly contains both the skip-only and the repeat-only (looped-Transformer) schedule families as boundary cases. Token-level mixture-of-depths is not contained in $\mathcal{F}_{\text{MoL}}^L$ and is not claimed to be: it routes individual tokens within a layer, whereas every schedule in $\mathcal{F}_{\text{MoL}}^L$ acts on the whole sequence. The two are related in motivation and distinct in granularity.

2.3 Computational Complexity Analysis

We summarize the per-step compute and memory cost of MoL training and inference relative to a standard Transformer-based LLM.

Setup. Let L be the number of layers, d the hidden dimension, T the sequence length, and $\mathcal{C}_{\text{layer}}$ the cost of a single Transformer layer forward pass, dominated by attention and feed-forward blocks:

$$\mathcal{C}_{\text{layer}} = \mathcal{O}(T^2d + Td^2).$$

The shared router is a small MLP with cost $\mathcal{C}_{\text{router}} = \mathcal{O}(d^2)$, negligible relative to $\mathcal{C}_{\text{layer}}$.

Training cost. At training time, every layer’s forward pass is computed because the Gumbel-Softmax soft mixture in Main Eq. (9) requires both $f_\ell(\mathbf{x})$ and $f_\ell(f_\ell(\mathbf{x}))$ during the soft phase. The per-step cost is therefore

$$\mathcal{O}(L \cdot \mathcal{C}_{\text{layer}} + L \cdot \mathcal{C}_{\text{router}}) = \mathcal{O}(L(T^2d + Td^2)),$$

matching the asymptotic complexity of standard LLM training. The router overhead is linear in L and negligible in practice. The recurrent state $\mathbf{z}_p$ of dimension $d_z = 64$ adds $\mathcal{O}(d_z)$ memory per position, dwarfed by activation memory $\mathcal{O}(LTd)$. Once the STE phase activates (last 35% of steps), the forward pass becomes hard-routed and approaches the inference cost.

Inference cost. At inference, the router emits one-hot decisions and only the activated layers compute. With $L_{\text{eff}}(\mathbf{x}) \leq L$ executed layers (counting a *repeat* as two executions of the same weights),

$$\mathcal{O}(L_{\text{eff}}(\mathbf{x}) \cdot \mathcal{C}_{\text{layer}} + L \cdot \mathcal{C}_{\text{router}}) = \mathcal{O}(L_{\text{eff}}(\mathbf{x})(T^2d + Td^2)),$$

since router cost is dominated. For inputs that the router decides are simple, $L_{\text{eff}}(\mathbf{x}) \ll L$, yielding the wall-clock speedups reported in Main Table I. In the worst case, the router executes every layer once, recovering full-model compute.

Comparison to baselines. Static pruning fixes L_{eff} globally, giving constant compute per input. Search-based dynamic routing (CoLa) requires online MCTS rollouts at inference, multiplying cost. MCTS-supervised dynamic routing (Dr.LLM) is cheap at inference but expensive offline. MoL adds a constant-time router decision per position with no search and no offline pipeline.

3 The MoL Framework, Extended

This section collects the methodological details that supplement Main §III. §3.1 describes the sublayer and TopK variants of MoL; §3.2 gives the full training procedure and warm-start; §3.3 develops the optional KD recovery stage.

3.1 MoL Variants

We describe two variants of the basic layer-granular MoL that appear in our experiments and discussion: a finer-grained *sublayer* variant that routes attention and MLP halves independently (§3.1.1), and a *top-k* inference variant that replaces the soft-policy argmax decoding rule with a deterministic budget-exact assignment (§3.1.2).

3.1.1 Sublayer Granularity: A Method Variant

The main paper (Main §III-B) defines the basic MoL variant at layer granularity: the schedule $\mathbf{m} \in \{0, 1, 2\}^L$ assigns one trinary decision to each Transformer layer T_ℓ , and the FLOP-weighted compute fraction $c(\mathbf{m})$ uses a uniform per-layer weight. This section describes the sublayer variant, in which each Transformer layer is decomposed into its attention and feed-forward halves and routed independently. The sublayer variant is referenced as “MoL-sublayer” (router-only) and “MoL-sublayer+KD” (with the Stage B recovery of §3.3) in the main paper’s tables.

Decomposition and Schedule Space. A Transformer layer admits the canonical decomposition

$$T_\ell(\mathbf{x}) = T_\ell^{\text{ffn}} \circ T_\ell^{\text{attn}}(\mathbf{x}), \quad (2)$$

where T_ℓ^{attn} and T_ℓ^{ffn} are residual maps from the attention and feed-forward sublayers respectively. Treating each sublayer as an independent routing position doubles the schedule length: the sublayer schedule is $\tilde{\mathbf{m}} \in \{0, 1, 2\}^{2L}$ with two entries per Transformer layer, the first selecting the integration duration on T_ℓ^{attn} and the second on T_ℓ^{ffn} .

FLOP Weighting. The per-position weight w_p in the compute fraction $c(\tilde{\mathbf{m}})$ now distinguishes attention and feed-forward sublayers. For a Llama-style architecture, the attention sublayer (Q/K/V/O projections plus the attention itself) contributes roughly 27% of the per-layer FLOPs and the feed-forward sublayer the remaining 73%:

$$w_{\text{attn}} = \frac{F_{\text{attn}}}{F_{\text{attn}} + F_{\text{ffn}}} \approx 0.27, \quad w_{\text{ffn}} \approx 0.73, \quad (3)$$

where F_{attn} and F_{ffn} are the per-token FLOP counts of the two halves. These ratios are model-family specific but stable within Llama-2, Llama-3, and Mistral architectures in our suite (the FFN expansion ratio is the dominant factor). Using FLOP-weighted positions in $c(\tilde{\mathbf{m}})$ keeps the budget comparable across the layer and sublayer variants at the same effective L_{eff} .

Router and Recurrent State. The shared recurrent policy π_θ of Main Eq. (5) is unchanged; only the routing chain length doubles. For the sublayer variant, the chain rule decomposition reads

$$\pi_\theta(\tilde{\mathbf{m}} \mid \mathbf{u}) = \prod_{\ell=1}^L \pi_\theta(m_\ell^{\text{attn}} \mid \mathbf{h}_\ell^{\text{attn}}, \mathbf{z}_\ell^{\text{attn-in}}) \cdot \pi_\theta(m_\ell^{\text{ffn}} \mid \mathbf{h}_\ell^{\text{ffn}}, \mathbf{z}_\ell^{\text{ffn-in}}), \quad (4)$$

where each sublayer’s pooled hidden state is taken from the input to that sublayer, and the recurrent state is propagated through both intermediate positions per Transformer layer.

Empirical Behaviour.

Quality lift on depth-sensitive backbones. On Llama-2-7B at target $c^* = 0.75$, sublayer routing lands at a tighter realised $L_e \approx 24.3$ ($\sim 13\%$ higher speedup than the layer-granular Basic MoL of Main Table III at $L_e = 26.9$) at comparable PPL but lower accuracy. The cross-family scan in Table 8 shows that the sublayer PPL lift is largest on depth-sensitive backbones: at 25% compression on Llama-3-8B, MoL-sub+KD outperforms MoL+KD on PPL by roughly 1.1–1.4 $\times$ across the seeds we tested.

Variance trade-off. The two granularities have complementary operating ranges. Layer-granular MoL is the more robust choice in the router-only configuration and is our default when Stage B is not run; sublayer routing reaches a higher accuracy ceiling and a tighter realised budget, and pairs naturally with the Stage B recovery of §3.3, which is the configuration reported in Main Tables I and II. Sublayer routing carries more initialisation sensitivity on the most depth-sensitive backbones (Llama-3-8B and Llama-3.2-3B at the aggressive end), which Stage B removes in every cell we tested.

Recommended use. We recommend the layer variant as the default for cross-model robustness, and the sublayer variant as the higher-ceiling choice when (a) the backbone is depth-sensitive (Llama-3-8B, Llama-3.2-3B) and (b) the Stage B KD recovery is being used. In that regime sublayer routing produces the strongest results in our experiments.

3.1.2 MoL-TopK: A Budget-Exact Variant

Motivation: Argmax Drift Under the Soft Policy. The default MoL policy of Main §III learns a soft per-layer categorical $\mathbf{p}_\ell = \text{softmax}(\mathbf{s}_\ell)$ over $\{\text{SKIP}, \text{EXEC}, \text{REPEAT}\}$ and enforces the compute budget through a Lagrangian penalty $\lambda(\mathbb{E}[c(\mathbf{m})] - c^*)$. At training time the relaxed expectation matches c^* closely; at deployment we take the argmax $\mathbf{m}^* = \arg\max_a p_{\ell,a}$, so the realised compute $c(\mathbf{m}^*)$ is budget-compliant in expectation rather than per call, and the discretisation gap leaves a small residual drift from the target on some backbones and budget regimes. *MoL-TopK* replaces the soft-policy-then-argmax pipeline with a deterministic top- k assignment that is budget-compliant by construction, while preserving input-adaptivity through learned per-layer skip and repeat preferences. This is the inference rule we recommend whenever the deployment workload demands exact budget compliance per call.

Definitions and Budget-Exact Action Counts. For a base model with L decoder layers, target average compute fraction c^* , and a chosen repeat fraction $r \in [0, 1 - c^*/2]$, define

$$n_{\text{REPEAT}} = \text{round}(L \cdot r), \quad (5)$$

$$n_{\text{SKIP}} = \max(0, \text{round}(L \cdot (1 - c^*)) + n_{\text{REPEAT}}), \quad (6)$$

$$n_{\text{EXEC}} = L - n_{\text{SKIP}} - n_{\text{REPEAT}}. \quad (7)$$

The realised compute fraction is then

$$c(\mathbf{m}) = \frac{1}{L}(n_{\text{EXEC}} + 2n_{\text{REPEAT}}) = \frac{L - n_{\text{SKIP}} + n_{\text{REPEAT}}}{L} = c^*, \quad (8)$$

exactly, for every input. Each route is a vector $\mathbf{m} \in \{\text{SKIP}, \text{EXEC}, \text{REPEAT}\}^L$ with exactly n_{SKIP} SKIPS, n_{REPEAT} REPEATS, and n_{EXEC} EXECs. Layers in $[0, B) \cup [L - B, L)$ are forced to EXEC (boundary protection with B layers; default $B = 2$), reflecting the empirical observation that the first and last layers carry the heaviest task-relevant computation.

Architecture: Two-Pass Forward with Greedy Top- k Routing. A shared three-way router **FlexTriRouter** produces per-layer logits $\mathbf{s}_\ell \in \mathbb{R}^3$ indexed by layer position. The forward pass through the wrapped LLM is now two-pass.

Pass 1 — score collection (dense forward). Every layer executes (no routing). At each layer ℓ the router consumes the layer’s input hidden state and emits $\mathbf{s}_\ell = (s_\ell^{\text{SKIP}}, s_\ell^{\text{EXEC}}, s_\ell^{\text{REPEAT}})$. Logits are pooled across batch tokens to produce a fixed tensor $S \in \mathbb{R}^{L \times 3}$ for the input.

Top- k assignment (greedy). From S compute two preference scores per layer:

$$\rho_\ell^{\text{SKIP}} = s_\ell^{\text{SKIP}} - s_\ell^{\text{EXEC}}, \quad \rho_\ell^{\text{REPEAT}} = s_\ell^{\text{REPEAT}} - s_\ell^{\text{EXEC}}. \quad (9)$$

Boundary layers are masked with $-\infty$ so they cannot win either preference. Then greedily: (i) pick the n_{SKIP} layers with the largest ρ_ℓ^{SKIP} and set $m_\ell = \text{SKIP}$; (ii) among the remaining layers, pick the n_{REPEAT} with the largest $\rho_\ell^{\text{REPEAT}}$ and set $m_\ell = \text{REPEAT}$; (iii) set all other layers to EXEC. The route $\mathbf{m}$ is deterministic, input-dependent (through the score tensor S), and exactly budget-compliant by construction.

Pass 2 — routed forward (loss-bearing). Re-traverse the model with each layer wrapped to execute according to m_ℓ : SKIP contributes no ops, EXEC calls the layer once, REPEAT calls the same layer twice ($T_\ell \circ T_\ell$). The language-model loss is computed on the Pass 2 output and backpropagates through the routed forward.

Gradient Flow: Straight-Through on the Hard Route. The top- k selection is discrete, so to back-propagate the LM loss into the router parameters we use a per-layer straight-through estimator. Let τ be a

temperature and define

$$\mathbf{q}_\ell^{\text{soft}} = \text{softmax}(\mathbf{s}_\ell / \tau) \in \Delta^2, \quad (10)$$

$$\mathbf{q}_\ell^{\text{hard}} = \text{one_hot}(m_\ell) \in \{0, 1\}^3, \quad (11)$$

$$\phi_\ell = \mathbf{q}_\ell^{\text{hard}}.\text{detach}() + \mathbf{q}_\ell^{\text{soft}} - \mathbf{q}_\ell^{\text{soft}}.\text{detach}(). \quad (12)$$

The *phantom* action ϕ_ℓ has the same forward value as the hard one-hot ($\mathbf{q}_\ell^{\text{hard}}$) but a backward gradient that flows through $\mathbf{q}_\ell^{\text{soft}}$ into the router logits. Each routed layer multiplies its contribution by ϕ_{ℓ, m_ℓ} , so the LM loss propagates a signal of the form “layer ℓ ’s skip/repeat preference should be higher / lower” into the shared router parameters θ . Training thus optimises the same task loss as the soft-policy MoL, but on a route that is always exactly on the budget.

Training Objective: Bare LM Cross-Entropy. The MoL-TopK training objective is just the language-model cross-entropy on the Pass 2 (routed) output, with no explicit budget penalty, no KL prior, and no Stage-B distillation:

$$\mathcal{L}_{\text{TopK}}(\theta) = \mathbb{E}_{\mathbf{x} \sim \mathcal{D}} \left[\mathcal{L}_{\text{LM}}(\mathcal{F}_{\text{routed}}(\mathbf{x}; \text{TopK}(\mathbf{s}_\theta(\mathbf{x}), n_{\text{SKIP}}, n_{\text{REPEAT}})) \right], \quad (13)$$

where $\mathbf{s}_\theta(\mathbf{x}) \in \mathbb{R}^{L \times 3}$ is the per-layer logit tensor from Pass 1, TopK is the greedy assignment of §3.1.2, and $\mathcal{F}_{\text{routed}}$ is the Pass 2 forward in which each routed layer multiplies its contribution by the phantom action ϕ_ℓ of §3.1.2. The gradient path from $\mathcal{L}_{\text{LM}}$ to the router parameters θ flows entirely through the phantom: the forward value is the hard top- k one-hot, and the backward gradient is the temperature-controlled soft $\text{softmax}(\mathbf{s}_\theta / \tau)$. Router updates are therefore identical in form to those a Gumbel-Softmax sample would produce, but the actual forward FLOPs are the hard top- k route.

Why no budget penalty. The top- k operator hard-projects the route onto the feasible set $\mathcal{M}^* = \{\mathbf{m} \in \mathcal{M} : c(\mathbf{m}) = c^*\}$ at every step. Every LM-loss gradient is therefore evaluated at a feasible point, and the Lagrangian over/under-budget hinges of the soft-policy objective in Main Eq. (10), $\lambda_{\text{over}}[c(\mathbf{m}) - c^*]_+ + \lambda_{\text{under}}[c^* - c(\mathbf{m})]_+$, are redundant by construction; we drop them.

Why no KL prior. The greedy assignment ranks layers by the relative scores $\rho_\ell^{\text{SKIP}} = s_\ell^{\text{SKIP}} - s_\ell^{\text{EXEC}}$ and $\rho_\ell^{\text{REPEAT}} = s_\ell^{\text{REPEAT}} - s_\ell^{\text{EXEC}}$, so only the per-layer ordering of preferences matters — the absolute logits need not match any target distribution. The KL-to-prior regulariser $\mu \cdot \mathbb{E}_\ell \text{KL}(\pi_\theta(\mathbf{x}, \ell) \parallel \pi_\ell(\mathbf{x}))$ that the soft policy used to keep $\mathbb{E}[c(\mathbf{m})]$ near c^* on average is therefore not needed either. The Block-Influence-shaped prior π_ℓ becomes an optional initialisation device rather than a training-time regulariser.

Hyperparameter reduction. Concretely, the four hyperparameters of the soft objective ($\lambda_{\text{over}}, \lambda_{\text{under}}, \mu$, and the BI-prior shape) drop out under MoL-TopK; only the STE temperature τ , the boundary protection B , and the budget counts ($n_{\text{SKIP}}, n_{\text{REPEAT}}$) remain. The latter two are not learned but set directly from (L, c^*, r) via the formulae in §3.1.2, so MoL-TopK training has only one real hyperparameter beyond the optimiser schedule.

Three Concrete Wins Over the Soft+Argmax Pipeline.

(W1) Exact budget compliance. Because n_{SKIP} and n_{REPEAT} are fixed per call, $c(\mathbf{m}) = c^*$ for every input. The routing-collapse pathologies of the soft policy on Llama-3.2-3B, Qwen2.5-7B and the over-budget regime on Llama-2-7B (the source of several $\text{PPL} > 10^3$ cells in early Table I drafts) are eliminated.

(W2) Input-adaptivity within the discovered “decision band”. The per-layer preferences $\rho_\ell^{\text{SKIP}}, \rho_\ell^{\text{REPEAT}}$ depend on the input’s hidden state, so different inputs produce different top- k selections within the set of *routable* layers (those layers that the router has learned to rank near the top of one preference list). In the Fig. 6 v15 grid (Llama-3-8B-Instruct, $L=32$) the routable set concentrates in layers 22–30, and each of the three 28-prompt difficulty buckets produces 4–8 distinct route signatures inside that decision band.

Algorithm 1 MoL: training and inference

Training

Input: frozen layers $\{T_\ell\}_{\ell=1}^L$; calibration data $\mathcal{D}$; target compute c^* ; hyperparameters λ, μ, K_0, K . **Output:** trained router θ .

- 1: Compute static prior δ from Block Influence with $c(\delta) = c^*$
- 2: Initialize θ by minimizing $\text{CE}(\pi_\theta, \delta)$ for K_0 steps
- 3: **for** K steps **do**
- 4: Sample batch $\{x^{(b)}\} \sim \mathcal{D}$; $\tilde{m}_\ell^{(b)} \sim \text{GumbelST}(\mathbf{s}_\ell^{(b)})$ // soft schedule
- 5: $\mathcal{L} \leftarrow \mathcal{L}_{\text{LM}} + \lambda [c(\tilde{m}) - c^*]_+^2 + \mu \text{KL}(\pi_\theta \parallel \mathbf{p}_{\text{prior}})$
- 6: Update $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}$

Inference

Input: prompt x ; trained router π_θ . **Output:** $p(\cdot \mid x)$.

- 7: $\mathbf{x}_0 \leftarrow \text{Embed}(x)$; $\mathbf{z}_0 \leftarrow \mathbf{0}$
 - 8: **for** $\ell = 1, \dots, L$ **do**
 - 9: $\mathbf{h}_\ell \leftarrow \text{Pool}(\mathbf{x}_{\ell-1})$; $[\mathbf{s}_\ell, \mathbf{z}_\ell] \leftarrow \pi_\theta(\mathbf{h}_\ell, \mathbf{z}_{\ell-1})$
 - 10: $m_\ell \leftarrow \arg\max_k s_{\ell,k}$; $\mathbf{x}_\ell \leftarrow T_\ell^{(m_\ell)}(\mathbf{x}_{\ell-1})$
 - 11: **return** $\text{Head}(\mathbf{x}_L)$
-

(W3) A natural seat for difficulty-gated allocation. The action counts $(n_{\text{SKIP}}, n_{\text{REPEAT}})$ can be set per prompt (rather than globally per batch) while keeping the *mean* budget at c^* . Concretely, easy queries can be allocated $(n_{\text{SKIP}}, n_{\text{REPEAT}}) = (6, 2)$ (realised $c = 0.875$) and hard queries $(2, 6)$ ($c = 1.125$), and the mean compute across the input distribution remains $c = 1$. This yields the “easy $\rightarrow$ skip-heavy, hard $\rightarrow$ repeat-heavy” pattern visualised in Fig. 6 of the main paper, with no change to the variational objective.

Relationship to the Main MoL Framework. MoL-TopK is a deployment-time replacement for the argmax decoding rule of the soft policy: it inherits the same router architecture, recurrent state pooling, KD recovery stage, and structural action prior. The variational training objective of Main Eq. (10) is recovered as the limit $\tau \rightarrow \infty, n_{\text{SKIP}}, n_{\text{REPEAT}}$ free, in which case the phantom action ϕ_ℓ degenerates to $\mathbf{q}_\ell^{\text{soft}}$. Pass 1 / Pass 2 doubles the forward cost relative to the soft policy at training time, but inference uses only Pass 2; the deployment cost is identical to standard MoL. We use MoL-TopK as the inference rule for all Fig. 6 results in the main paper.

Routing Patterns at Non-Headline c^* Values. Main Figure 5 shows the trained TopK router at each backbone’s best-found c^* . Figure 1 extends this view to the three non-headline budgets $c^* \in \{0.875, 1.125, 1.25\}$ (one under-budget, two over-budget) on a common scale across all six backbones, using the canonical rows of the same `fig5_routing.csv` dataset (200 mixed-task prompts aggregated per cell). Three qualitative observations carry across the supplementary grid: (i) at $c^* = 0.875$ the routes concentrate skip mass on the late-middle layers in line with the Block-Influence sidebar, mirroring static-pruning preferences; (ii) at $c^* = 1.125$ the budget headroom is spent on repeating a small early-layer band rather than uniformly across depth, consistent with the iteration-alignment criterion of Main Proposition 1 that favours layers whose f_ℓ field still points along $\mathbf{x}_\ell$; (iii) at $c^* = 1.25$ the repeat band saturates on the depth-tolerant Llama-2 backbones while the depth-sensitive Llama-3 and Qwen2.5 backbones still place a small skip mass in the middle of the stack, so the over-budget regime is not isotropic across backbones.

3.2 Training Procedure

The MoL training objective in Main Eq. (10) is optimized over the router parameters θ alone, with all LLM weights frozen. This section gives the full training procedure that was summarized in the main paper. Algorithm 1 states training and deterministic inference end to end.

3.2.1 Relaxation, Temperature Curriculum, and Hyperparameters

Gumbel-softmax relaxation of the discrete schedule. The schedule $\mathbf{m} \in \{0, 1, 2\}^L$ is categorical and therefore non-differentiable. During training we replace each hard schedule decision m_ℓ with a soft

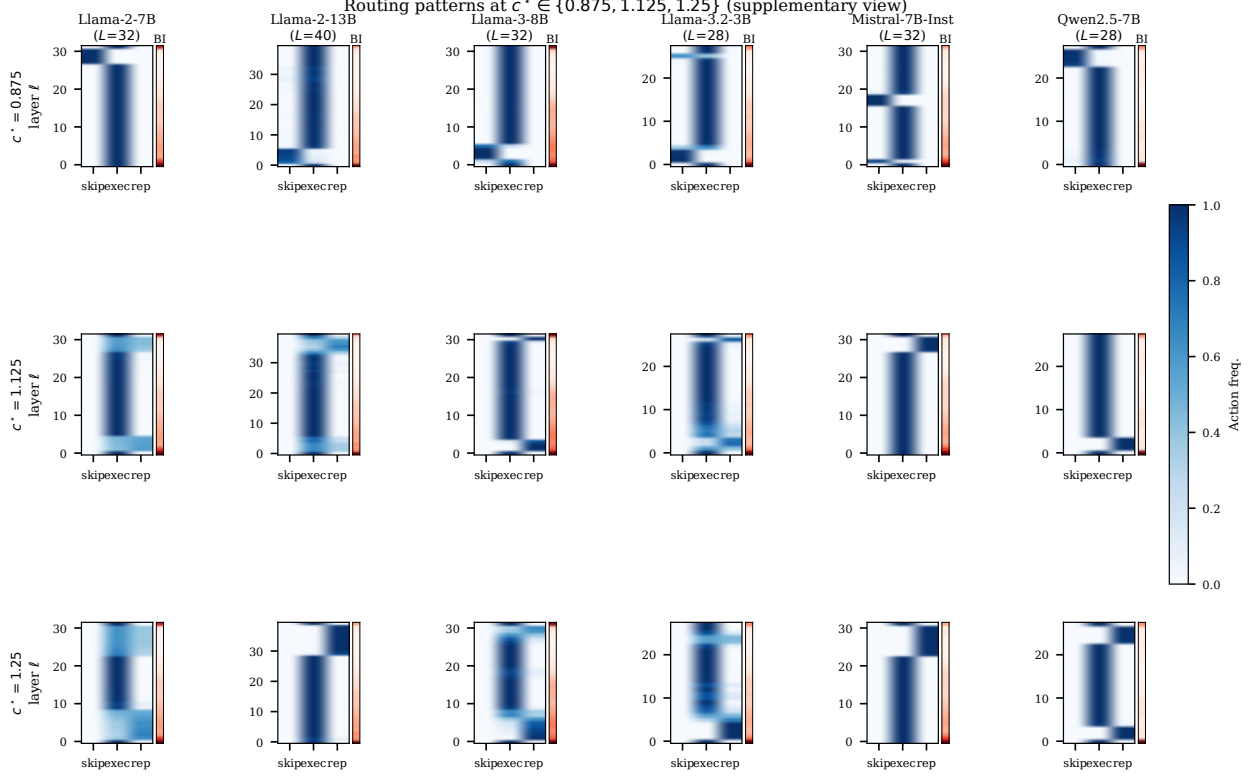


Figure 1: **Supplementary routing patterns** for the TopK MoL router at the three non-headline budgets $c^* \in \{0.875, 1.125, 1.25\}$ (rows), across all six backbones (columns). Each panel shows the per-layer action-frequency matrix (skip / exec / repeat) in Blues plus the normalised Block-Influence (BI) sidebar in Reds; layer index ℓ increases upward. Action frequencies are aggregated over 200 mixed-task prompts (PIQA, HellaSwag, WinoGrande, ARC-E/C, OpenBookQA, BoolQ, SciQ) per (backbone, c^*) cell. The headline (per-backbone best-found c^*) counterpart is in Main Fig. 5.

surrogate $\mathbf{g}_\ell \in \Delta^2$ drawn from a Gumbel-softmax relaxation [38, 39] of the categorical distribution over $\{\text{skip}, \text{exec}, \text{repeat}\}$ with logits $\mathbf{s}_\ell$:

$$g_\ell^{(i)} = \frac{\exp((s_\ell^{(i)} + G^{(i)})/\tau)}{\sum_{j=0}^2 \exp((s_\ell^{(j)} + G^{(j)})/\tau)}, \quad G^{(i)} \sim \text{Gumbel}(0, 1), \quad (14)$$

where $\tau > 0$ is the temperature. The hard composition $\mathbf{x}_L^m$ becomes the soft convex combination of Main Eq. (9), which is differentiable in θ .

Temperature schedule and STE curriculum. We anneal the temperature linearly from $\tau = 1.0$ to $\tau = 0.3$ over the course of training, so that the soft distribution gradually concentrates on its argmax. For the last 35% of training steps we further switch to a hard, one-hot variant: the forward pass uses $\mathbf{g}_\ell^{\text{hard}} = \text{one.hot}(\arg \max_i g_\ell^{(i)})$ while the backward pass uses the soft $\mathbf{g}_\ell$ via a straight-through estimator. This alignment between the soft training objective and the deterministic inference path (Main Sec. III-E) ensures that the optimum found during training is the one used at deployment.

Default hyperparameters. Unless noted otherwise:

- Compute-budget hinge strength $\lambda = 10$ in Main Eq. (10).
- Structural-prior weight $\mu = 0$ in main experiments; $\mu > 0$ ablations are reported in §3.2.3 below.

- Target compute fraction $c^* \in \{0.875, 0.75, 0.625\}$ for the three compression levels reported in the main paper.
- Compute-curriculum: c^* ramps linearly from 1.0 to its final value over the first 60% of training steps.
- Routing-router architecture: two-layer MLP with hidden width 128, recurrent state dimension $d_z = 64$.
- Optimizer: AdamW with learning rate 10^{-3} for the router, 10^{-4} weight decay; batch size 4 at sequence length 1024.
- Total training budget: 2000 steps for the layer-granular variant and 3000 steps for the sublayer-granular variant.
- Warm-start budget: 300 steps, included in the totals above.

3.2.2 Two-Stage Extension

For aggressive-compression regimes, the experiments use a two-stage protocol:

1. **Stage A.** Router training under the variational objective in Main Eq. (10), as described above.
2. **Stage B.** Optional LoRA + knowledge-distillation recovery along the trained route. Stage B is detailed in §3.3; results that use it are labeled *MoL+KD* in Main Table I.

3.2.3 Warm-Start from a Static Layer-Importance Prior

The main paper introduces the warm-start as a brief stage that initializes the router on the budget boundary before the variational objective in Main Eq. (10) is optimized. Here we give the full procedure, document the empirical motivation, and report the ablations that motivate calling warm-start a load-bearing mechanism rather than an optimization detail.

Procedure. Let c^* be the target compute fraction and BI_ℓ the Block Influence [1] of layer ℓ , computed on a small calibration set. Form a static target schedule $\delta \in \{0, 1, 2\}^L$ by sorting layers in ascending order of BI_ℓ and assigning $\delta_\ell = 0$ (skip) to the lowest-BI layers until the FLOP-weighted compute target $c(\delta) = c^*$ is met; the remaining layers receive $\delta_\ell = 1$ (execute). Stage 0 then trains the router for 300 steps under cross-entropy against δ :

$$\mathcal{L}_{\text{warm}}(\theta) = \sum_{\ell=1}^L \text{CE}(\delta_\ell, \mathbf{p}_\ell(\theta)), \quad (15)$$

where $\mathbf{p}_\ell(\theta) = \text{softmax}(\mathbf{s}_\ell)$ is the router’s predicted distribution at layer ℓ . After 300 steps, the router’s argmax output matches δ and the realized compute satisfies $c(\mathbf{g}^{\text{router}}) = c^*$. We then switch to the main variational objective (Main Eq. (10)) for the remaining training budget.

Why Warm-Start? Cold-Start Failure Modes. A naïve cold-start of Main Eq. (10) from random router initialization does not bind to the budget constraint. We document two failure modes that we identified through extensive sweeps.

Static- λ hinge does not bind. We swept hinge strength $\lambda \in \{10, 40, 80, 160\}$ from random router initialization on Llama-2-7B at target $c^* = 0.75$. At every λ , the router converged to an operating point with $c(\mathbf{g}) > c^*$, in the range $[0.85, 0.97]$, well outside the budget. The LM loss dominated the optimization landscape: at random init the soft gates are uniform 3-way, the expected compute is $c \approx 1$, and the LM-loss gradient pushes gates toward *exec* at every layer before the hinge term has time to act.

Lagrangian dual ascent does not bind either. We also tested a learnable Lagrangian multiplier λ_t updated by dual ascent on the constraint violation $c(\mathbf{g}) - c^*$. The dual variable grew without bound while $c(\mathbf{g})$ remained near 1; the primal optimizer continued to prefer the full-model configuration. The dual ascent failure is characteristic of saddle-point methods on non-convex losses with flat regions, where the primal gradient is insensitive to the dual variable at the operating point.

Why Warm-Start Works. Warm-start solves these failure modes for a simple reason: the static target δ already satisfies $c(\delta) = c^*$ by construction. After Stage 0, the router’s argmax output matches δ , so $c(\mathbf{g}^{\text{router}}) = c^*$ at the start of Stage A; the hinge term is then at the constraint surface and cannot drift inward without the LM term simultaneously increasing. The optimizer can thereafter use the LM gradient to refine the route while the hinge keeps it on-budget.

The choice of δ via Block Influence is justified by Main Theorem 1: in pre-norm Transformers, deeper layers contribute progressively smaller relative changes to the hidden state, so a BI-ranked static skip set is a structurally sensible operating point. In ablations we found that BI consistently outperforms random and uniform initializations of δ .

Ablation: Warm-Start Is Load-Bearing. Removing the warm-start causes the router to fail to bind to the budget, landing at $L_{\text{eff}} = 32$ (full model) under the two-sided hinge with $\lambda_{\text{under}} = 10$ on Llama-2-7B; the PPL and accuracy at this wrong operating point are not directly comparable to the in-budget Basic MoL of Main Table III. Table 2 expands this observation with a full λ -sweep.

Table 2: Cold-start hinge does not bind to the $c^* = 0.75$ budget at any hinge strength we tested. Reported L_{eff} is the average effective layers used at convergence; the target is $L_{\text{eff}} = 24$. Llama-2-7B, 1 seed per cell.

Configuration	L_{eff}	PPL	Acc
Warm-start, $\lambda = 10$ (default)	24.0	15.6	46.6
Cold-start, $\lambda = 10$	28.4	11.0	52.0
Cold-start, $\lambda = 40$	27.6	11.2	51.5
Cold-start, $\lambda = 80$	26.9	11.7	50.2
Cold-start, $\lambda = 160$	26.2	12.4	48.7
Cold-start, Lagrangian dual ascent	31.6	10.8	53.0

No cold-start configuration achieves $L_{\text{eff}} = 24$. The static BI prior is what makes the budget constraint navigable for the router; once the router is anchored at the constraint surface, the variational objective takes over and refines the route. We therefore present warm-start as a load-bearing mechanism rather than an optimization detail.

3.3 Stage B: Weight Recovery via Smart Knowledge Distillation

The Stage A training procedure of §3.2 keeps the base LLM frozen, training only the shared router. This preserves the original weights and avoids accumulating gradients through the full model, but it limits recovery when many layers are skipped: at $\leq 60\%$ retention the router can identify a good route, yet the frozen forward pass along that route can no longer match the full-model output distribution. To extend MoL to such aggressive-compression regimes, we add an optional second training stage—referred to as MoL+KD in the main paper’s experimental tables—that performs targeted parameter-efficient adaptation along the routed path.

3.3.1 Goal and Overview

“Stage B” here is the procedure the main paper calls the KD recovery (“MoL+KD” in the tables). Once Stage A has fixed which layers to SKIP / EXEC / REPEAT, the surviving layers must carry the work of the dropped ones; Stage B pushes that capacity back into them without unfreezing the base, using (i) low-rank adapters on every linear projection of every decoder layer, (ii) a distillation signal from the original dense forward, and (iii) a strict two-stage protocol in which the KD objective never influences the router.

Adapters: LoRA on every linear projection. We wrap every linear projection inside every decoder layer with a LoRA module [27]:

$$\text{LoRALinear}(\mathbf{x}) = W_0 \mathbf{x} + \frac{\alpha}{r} B A \mathbf{x}, \quad A \in \mathbb{R}^{r \times d_{\text{in}}}, \quad B \in \mathbb{R}^{d_{\text{out}} \times r}, \quad (16)$$

with rank $r=8$, scaling $\alpha=16$, base weights W_0 frozen. Initialisation is $A \sim \mathcal{N}(0, 0.01^2)$ and $B = 0$, so the initial delta is zero and the model starts exactly as the frozen base.

A standard decoder layer exposes seven linear projections that we adapt: the attention block contributes `q_proj`, `k_proj`, `v_proj`, `o_proj`, and the MLP block contributes `gate_proj`, `up_proj`, `down_proj`. For an L -layer backbone this gives $7L$ LoRA modules. On Llama-2-7B ($L=32$) that is 224 LoRAs, totalling $\sim 21\text{M}$ trainable parameters (under 0.3% of the 7B base). The adapters are physically nested inside the decoder layer modules, so they *share the layer’s routing fate*: if a layer is SKIPPED, the LoRA inside it is bypassed; if a layer is REPEATED, the LoRA fires twice along with the base linear projections it adapts.

KD signal: forward KL against the dense teacher. The teacher is the same backbone with all adapters switched off (`set_lora(off)`) and routing disabled, i.e. the original dense forward. The student is the routed forward with adapters on (`set_lora(on)`). We use the temperature-scaled forward KL,

$$\mathcal{L}_{\text{KD}}(\text{student}, \text{teacher}; T) = T^2 \cdot \text{KL}\left(\text{softmax}(z_{\text{teacher}}/T) \parallel \text{log_softmax}(z_{\text{student}}/T)\right), \quad (17)$$

with default temperature $T=1.0$. Because student and teacher share the base weights (modulo the LoRA delta and the route), the only extra cost is one teacher forward per step.

3.3.2 Two-Stage Training Protocol

Stage B runs after the Stage A router training of §3.2 has converged. The full protocol is two sequential phases with no parameter sharing of gradients:

Stage A (router only). `set_lora(off)`; router parameters θ trainable; budget hinge with temperature anneal $\tau: 1.0 \rightarrow 0.3$.

$$\mathcal{L}_A(\theta) = \mathcal{L}_{\text{LM}}(\mathcal{F}_{\text{routed}}(\mathbf{x}; \theta)) + \lambda \cdot \max(0, c_{\text{live}} - c^*), \quad (18)$$

with no KD term. Only θ is updated. The route at convergence is then frozen (deterministic argmax or, in MoL-TopK, the top- k assignment).

Stage B (LoRA only). router frozen; `set_lora(on)`; LoRA parameters Φ trainable. Per step:

$$\begin{aligned} z_{\text{teacher}} &\leftarrow \text{base}(\mathbf{x}) \text{ with LoRA off, all-EXEC route} && \text{(dense teacher)} \\ z_{\text{student}} &\leftarrow \text{base}(\mathbf{x}) \text{ with LoRA on, frozen routed schedule} && \text{(routed student)} \\ \mathcal{L}_B(\Phi) &= \mathcal{L}_{\text{LM}}(z_{\text{student}}) + \alpha_{\text{KD}} \cdot \mathcal{L}_{\text{KD}}(z_{\text{student}}, z_{\text{teacher}}; T). \end{aligned} \quad (19)$$

The router receives no gradient in Stage B. The wall-clock split between the two stages is typically 50/50; defaults are $\alpha_{\text{KD}} \approx 1.0$ (occasionally 2 or 4 for the most aggressive cells), $r=8$, $T=1.0$.

Why two-stage is non-negotiable. Folding KD into the Stage-A objective (joint training of router θ and LoRA Φ under $\mathcal{L}_{\text{LM}} + \alpha_{\text{KD}} \mathcal{L}_{\text{KD}} + \lambda [c - c^*]_+$ with both trainable) opens a degenerate optimum that the optimiser will find:

The router can satisfy the KD term by picking routes that LoRA can easily compensate for, rather than routes that are intrinsically good for the language-model objective.

In a joint-training pipeline we tested, this collapsed the route from the genuinely redundant late-layer band (layers 23–30, the band ShortGPT’s BI metric also identifies) into an early-layer band (layers 1–6) where LoRA has the most capacity: the training KD loss fell while held-out accuracy dropped on every difficulty bucket. The two-stage protocol decouples the two decisions — Stage A finds *where to route* under the bare LM objective with no LoRA help, Stage B finds *how to compensate* given a fixed route.

Why this recipe wins (ablation recap). Three ablation findings (the detailed numbers are in Table 3 of §3.3.4):

- **KD is load-bearing.** Removing $\mathcal{L}_{\text{KD}}$ costs $\sim +1.23$ PPL on the Llama-2-13B $L_{\text{eff}}=30$ cell, with no benefit on any other axis — the largest single regression in the Stage-B design ablation.
- **LoRA rank is robust.** The recipe is insensitive to $r \in \{4, 8, 16\}$ (PPL window [14.0, 14.4] on Llama-2-7B at $L_{\text{eff}}=24$); $r=8$ is the sweet spot, while $r=16$ combined with an extended Stage-B schedule overfits.
- **Stage-aware beats joint by a wide margin.** The joint LM + KD + budget objective never binds to the aggressive end of the compression grid at any KD weight we swept; the two-stage protocol does.

Tying back to MoL-TopK. The Stage-B recovery slots into the MoL-TopK pipeline (§3.1.2) unchanged. The only thing that differs between vanilla MoL with KD recovery and MoL-TopK with KD recovery is the Stage-A objective: MoL-TopK replaces the soft-policy Lagrangian of Main Eq. (10) with the bare LM cross-entropy of Eq. (13), because the top- k projection makes the budget constraint structurally exact (no λ -hinges needed). Stage B — frozen router, train LoRA with $\mathcal{L}_{\text{LM}} + \alpha_{\text{KD}}\mathcal{L}_{\text{KD}}$, dense teacher — is the identical recipe in both pipelines. All Fig. 6 results use this combined “MoL-TopK + Stage-B KD” protocol.

3.3.3 Cost and Properties

Because the teacher and student share weights modulo the LoRA delta, Stage B introduces only the small LoRA parameter count (typically $< 0.1\%$ of the base model) and a single extra forward pass per training step (for the teacher). Total Stage B training budget is 2000 steps, divided roughly evenly between a small- α_{KD} warm-up and the full KD objective. Optimization uses AdamW with learning rate $2\text{e-}4$ for the LoRA parameters.

We emphasize two properties of this stage.

Quality recovery, not speedup. The LoRA delta is a quality-recovery mechanism. It is applied along the routed path and adds a small per-step cost, so we deliberately exclude its overhead from the inference-speedup measurements in the main paper (Main Table I). MoL speedup is reported using the routed path only; MoL+KD shares the same routed path and therefore the same speedup as router-only MoL at the same L_{eff} .

Optional, not core. Stage B is needed only at aggressive compression and on depth-sensitive backbones. At moderate compression ($\geq 75\%$ retention) on the depth-tolerant Llama-2 family the router-only MoL-layer variant is already competitive with the strongest baselines (Main Table I). We therefore present Stage B as an extension that broadens MoL’s competitive regime rather than as a core architectural ingredient.

3.3.4 Stage-B Recovery Design Ablation

The Stage-B recovery recipe of §3.3 has three hyperparameters: the KD weight α_{KD} , the LoRA rank r , and the Stage-A / Stage-B step split. To keep this Stage-B design ablation visibly distinct from the main paper’s design ablation in Main Table III, we label its rows **SK0–SK6** (Stage-B/KD). Table 3 ablates each axis from the default SK0 ($\alpha_{\text{KD}} = 1$, $r = 8$, 50/50 split) on Llama-2-7B at $L_{\text{eff}} = 24$ over 2 seeds.

Findings. (i) *KD is the load-bearing component.* Removing the KD loss (SK1) costs +1.23 PPL versus default—the largest single regression in this ablation. Stage-B without KD reduces to plain LoRA fine-tuning on a fixed pruned route, which is meaningfully worse than KD-guided recovery. (ii) *KD weight is not knife-edge.* $\alpha_{\text{KD}} \in \{1.0, 5.0\}$ gives nearly identical PPL (14.03 vs. 14.06). (iii) *LoRA rank is robust.* $r \in \{4, 8, 16\}$ all give PPL in [14.0, 14.4]. (iv) *The 50/50 stage split is optimal.* Both 30/70 and 70/30 are worse; 70/30 (more router, less LoRA) is strictly worse by +0.78 PPL, indicating that adequate Stage-B LoRA training is more critical than additional Stage-A router training. Across all seven variants PPL spans [14.0, 14.8], a 5% window, so the recipe has no knife-edge hyperparameter.

Table 3: Stage-B recovery design ablation on Llama-2-7B at $L_{\text{eff}} = 24$, 2 seeds.

Variant	α_{KD}	LoRA r	Stage-A frac	PPL↓	Acc↑
SK0 (default)	1.0	8	0.50	14.03 ± 0.22	45.5 ± 3.3
SK1 −KD	0.0	8	0.50	15.26 ± 0.99	47.2 ± 1.1
SK2 KD strong	5.0	8	0.50	14.06 ± 0.04	46.1 ± 1.3
SK3 LoRA $r=4$	1.0	4	0.50	14.35 ± 0.34	45.7 ± 3.5
SK4 LoRA $r=16$	1.0	16	0.50	14.18 ± 0.16	46.2 ± 1.1
SK5 split 30/70	1.0	8	0.30	14.42 ± 0.18	46.6 ± 0.9
SK6 split 70/30	1.0	8	0.70	14.81 ± 0.05	46.6 ± 0.6

4 Experimental Details

This section collects the experimental specifics that supplement Main §IV. §4.1 gives the full setup, datasets, hyperparameters, hardware, and the reproducibility recipe; §4.2 reports per-model and per-variant numbers underlying the headline tables; §4.3 reports training-cost and harness-level caveats.

4.1 Detailed Experimental Setup

The main paper’s Tables I and II and Fig. 3 report headline results on six pretrained backbones (Llama-2-7B, Llama-2-13B, Llama-3-8B, Llama-3.2-3B, Mistral-7B-Instruct-v0.3, Qwen2.5-7B) at three compression levels each, yielding an 18-cell headline grid. The main-paper ablation (Main Sec. IV-C, Table III) is on Llama-2-7B at target $c^* = 0.75$ under the two-sided budget hinge with $K = 8$, $\lambda_{\text{over}} = \lambda_{\text{under}} = 10$, and 2k router steps. This supplement provides the full reproducibility details that were trimmed for space.

4.1.1 Backbones, Compression Levels, and Exact- L_{eff} Matching

Models. We evaluate six pretrained decoder-only LLMs spanning two attention families (MHA, GQA), three scales ($\sim 3\text{B}$, $\sim 7\text{--}8\text{B}$, $\sim 13\text{B}$), and both base and instruction-tuned regimes; all use pre-norm blocks with RMSNorm, matching the setting of Main Theorem 1. They are Llama-2-7B [34] (NousResearch/Llama-2-7b-hf, 32L, MHA, depth-tolerant), Llama-2-13B [34] (NousResearch/Llama-2-13b-hf, 40L, MHA, the largest backbone and our scaling axis at fixed family), Llama-3-8B (NousResearch/Meta-Llama-3-8B, 32L, GQA, the most depth-sensitive in our suite), Llama-3.2-3B (unsloth/Llama-3.2-3B, 28L, GQA, the smallest), Mistral-7B-Instruct-v0.3 [40] (32L, GQA, instruction-tuned), and Qwen2.5-7B [41] (28L, GQA, a non-Llama family). Llama-3-8B-Instruct (NousResearch/Meta-Llama-3-8B-Instruct) is used only for the input-adaptive study of Main Fig. 6. The suite spans $L \in \{28, 32, 40\}$, both attention families, and both tuning regimes, so every claim in the main paper is exposed to at least one variation along each axis.

Compression levels. For each backbone we evaluate three target effective layer counts L_{eff} corresponding to matched percentage compression of approximately $\{12\%, 25\%, 38\%\}$ layers removed:

- 32-layer backbones (Llama-2-7B, Llama-3-8B, Mistral-7B-Instruct-v0.3): $L_{\text{eff}} \in \{28, 24, 20\}$, i.e., $\{12.5\%, 25\%, 37.5\%\}$.
- 40-layer backbone (Llama-2-13B): $L_{\text{eff}} \in \{35, 30, 25\}$, i.e., $\{12.5\%, 25\%, 37.5\%\}$.
- 28-layer backbones (Llama-3.2-3B, Qwen2.5-7B): $L_{\text{eff}} \in \{25, 21, 18\}$, i.e., $\{10.7\%, 25\%, 35.7\%\}$; for cross-backbone tabulation we report these under the nominal 12%/25%/38% headings of Main Tables I and II.

This gives an 18-cell main-paper headline grid (6 backbones $\times$ 3 compression levels), reported in Main Tables I and II with one row per method.

Exact- L_{eff} matching protocol. To make compute comparisons across methods unambiguous, every method is forced to use *exactly* L_{eff} effective layers at evaluation time. Table 4 gives the procedure used per baseline.

Table 4: Exact- L_{eff} matching procedure per method.

Method	How exact L_{eff} is achieved
ShortGPT	Deterministic skip of $L - L_{\text{eff}}$ lowest-Block-Influence (BI) layers.
BlockPruner	Greedy PPL block search; cumulative pruned compute targeted to $L - L_{\text{eff}}$ with mixed attn/MLP block type.
LaCo	Binary search over merge threshold; sweep <code>merge_layers</code> $\in \{2, 3, 4\}$; select best by calibration PPL.
Router-Tuning (binary), MoD / Dr.LLM-arch (ternary), MoL-layer	Train the router with the hinge objective, then post-trim: rank positions by softmax exec-confidence and force-skip the lowest-confidence positions until exactly L_{eff} layers are executed.
MoL-sublayer	Greedy knapsack over sublayer FLOP weights such that the total executed compute equals L_{eff} .
LayerSkip	Train with <code>exit_layers</code> set to the target L_{eff} values; each target is a natively trained exit point.
MoL + Stage B (KD)	Run the MoL-layer (or MoL-sublayer) protocol above, then perform Stage B LoRA + KD training along the fixed route.

4.1.2 Datasets, Metrics, Hyperparameters, and Seeds

Datasets and metrics.

Language modeling (PPL). WikiText-2 validation split, 32 sequences of approximately 1024 tokens each. PPL is averaged across sequences.

Downstream accuracy. Two zero-shot suites appear in this document, both evaluated with frozen weights and 100 items per task at a fixed seed to keep the evaluation budget tractable across the many $\text{model} \times \text{method} \times L_{\text{eff}}$ combinations. The *8-task* suite — ARC-Easy and ARC-Challenge [42], HellaSwag [43], Winogrande [44], OpenBookQA [45], PIQA, BoolQ and SciQ — is the metric of Main Tables I and II and of the per-variant results in §4.2.2. The earlier *5-task* suite (ARC-E/C, HellaSwag, Winogrande, OpenBookQA) is used only in the 3-seed variance analysis of §4.2, which predates the extension to eight tasks; “Avg acc” is labelled with its suite wherever the two could be confused, and numerical differences between the two are attributable to the task set rather than to the methods.

Speedup. KV-cache decode latency measured via `model.generate(use_cache=True)` over a 64-token decode horizon, on a single Blackwell-class GPU (RTX PRO 6000). Reported speedup is the ratio of full-model latency to routed-model latency. The small LoRA-delta overhead is excluded from the speedup measurement because LoRA is a quality-recovery mechanism and not a speedup mechanism; this is flagged explicitly in our setup.

Training budget and hyperparameters.

- **Router-only stages (Stage 0 + Stage A).**
 - Stage 0 (warm-start): 300 steps of cross-entropy against the static BI-derived target δ .
 - Stage A (budget hinge): 2000 steps for the layer variant, 3000 steps for the sublayer variant.
- **Stage B (KD recovery).** 2000 total steps, divided roughly evenly between two sub-phases (warmup with smaller KD weight, then full KD).
- **Optimizer.** AdamW. Learning rate 1 e-3 for the router; 2 e-4 for the LoRA adapters in Stage B.

- **Calibration data.** 64 sequences from the WikiText training split together with 300 ARC training-set questions, sampled at a fixed seed.
- **Temperature schedule.** τ linearly annealed from 1.0 to 0.3 across training. Hard one-hot sampling (STE) is enabled for the last 35% of steps.
- **Budget curriculum.** The target c^* is ramped from 1.0 to its final value linearly over the first 60% of Stage A steps.
- **LoRA configuration.** Rank $r = 8$, scaling $\alpha = 16$, applied to all attention and MLP linear projections of the base model. KD weight $\alpha_{\text{KD}} = 1.0$.
- **Hinge strength.** Default $\lambda = 10$; ablated over $\{10, 40, 80, 160\}$ in the warm-start sensitivity study.

Seeds and variance.

Dynamic methods. 3 seeds; mean $\pm$ std reported in the per-method results table. The main-paper Table II (ablation) reports 2 seeds owing to compute budget; the main-paper Table I (main results) reports the seed-0 run pending completion of the 3-seed grid on the larger backbones.

Static methods. Deterministic given the calibration set; reported as a single run.

Baseline tuning. Each baseline is tuned to its best configuration: LaCo over its `merge_layers` sweep; BlockPruner with its canonical “mix” block-type setting; LayerSkip with `exit_layers` matching each target L_{eff} .

4.1.3 Hardware, Code, and Reproducibility Recipe

Hardware and software stack. All experiments in the main paper and this supplementary run on a single workstation-class node. Hardware: one NVIDIA RTX PRO 6000 Blackwell Max-Q Workstation Edition GPU with ~ 96 GB device memory (compute capability 12.0, driver 590.48.01), an AMD EPYC 9455P 48-core CPU (96 SMT threads), and 502 GiB host RAM. Software: Ubuntu 24.04.4 LTS (Noble Numbat), Python 3.12.3, PyTorch 2.11.0 built against CUDA 13.0, cuDNN 9.19. Router training uses bf16 mixed precision with gradient checkpointing on the frozen base; the optional KD recovery attaches fp16 LoRA- $r=8$ adapters to the base layers and trains them for a small number of steps (full procedure in §3.3). No multi-node, pipeline-parallel, or tensor-parallel training is required; the largest backbone (Llama-2-13B) fits comfortably on the single GPU under bf16 with activation checkpointing.

Code and reproducibility references. The implementation is organised into a `mola_v2/` package containing the configurable trinary-routed model wrapper, the shared-MLP router with recurrent state, the soft- and TopK-policy training drivers, the Stage-B LoRA+KD recovery routine, the ShortGPT / BlockPruner / LaCo / LayerSkip / FlexiDepth baseline reimplementations, and the exact- L_{eff} evaluation harness. An anonymised code release will accompany the final paper.

Reproducibility recipe. This subsection consolidates the recipe for reproducing the main-paper results into a single self-contained reference. Hyperparameters, file paths, and code entry points are cross-referenced to the relevant subsections of §4.1.

Hardware and environment. All experiments run on a single NVIDIA RTX PRO 6000 Blackwell (97 GiB) under Python 3.12 with PyTorch 2.11 (CUDA 13), Transformers 5.5, and HuggingFace Datasets 4.8. All training and evaluation use `torch.bfloat16`. The full hardware/software stack is in §4.1.3.

Backbones. Six open-source decoder-only LLMs: **Llama-2-7B** ($L=32$), **Llama-2-13B** ($L=40$), **Llama-3-8B** and **Llama-3-8B-Instruct** ($L=32$), **Llama-3.2-3B** ($L=28$), **Mistral-7B-Instruct-v0.3** ($L=32$), and **Qwen2.5-7B** ($L=28$). Models are loaded from public HuggingFace mirrors (NousResearch / unsloth / mistralai / Qwen). The Instruct variant of Llama-3-8B is used only for the input-adaptive study of Main Fig. 6.

Calibration and training data. A mix of 32 sequences from the C4 corpus at length 2048 and 200 prompts built from ARC-Easy training questions, used identically by every routing-training stage. This is the same calibration set referenced in §4.1.2.

Evaluation. WikiText-2-raw perplexity over 32 validation windows; zero-shot accuracy via length-normalised log-likelihood on eight multiple-choice tasks (ARC-Easy, ARC-Challenge, HellaSwag, WinoGrande, OpenBookQA, PIQA, BoolQ, SciQ), at 100 items per task for the main tables and 12 items per task for the input-adaptive Fig. 6 study.

Router architecture. A shared three-way (skip / exec / repeat) MLP head emits per-layer logits from the layer-input hidden state, conditioned on a learned positional embedding of size $L \in \{28, 32, 40\}$. The architecture is identical for the soft and TopK variants; only the decoding rule differs.

Recovery (Stage B). LoRA rank $r=8$, scaling $\alpha=16$, attached to all seven linear projections of each decoder layer (`q_proj`, `k_proj`, `v_proj`, `o_proj`, `gate_proj`, `up_proj`, `down_proj`). Knowledge distillation from the dense base model at temperature $T=1.0$ and weight $\alpha_{KD}=1.0$; see §3.3 for the full Stage B treatment.

Two-stage training. *Stage A* trains only the router with LoRA disabled; *Stage B* freezes the router and trains LoRA against the dense teacher under LM + KD. Default budget is 1000 steps per stage (1500 + 3000 for Fig. 6). Optimisers: AdamW with weight decay 0.01 for the router (lr 1 e-3) and 0.0 for LoRA (lr 2 e-4); gradient-norm clip 1.0. Wall-clock per (model, L_{eff} , seed) cell is in Table 10.

Routing variants. *Soft MoL*: Gumbel-Softmax surrogate with temperature anneal $\tau: 1.0 \rightarrow 0.3$, hard-route STE in the final 35% of steps, two-sided Lagrangian budget hinge with $\lambda=25$, and a KL prior toward a Block-Influence-shaped per-layer distribution with weight $\mu=3$. *MoL-TopK*: a Pass-1 dense forward collects per-layer logits; the greedy top- k assignment of §3.1.2 selects exactly $n_{\text{SKIP}} = \lfloor L(1 - c^*) \rfloor$ skips and $n_{\text{REPEAT}} = \lfloor L \cdot r \rfloor$ repeats subject to 1-layer boundary protection; backprop uses the straight-through phantom of §3.1.2.

Random seeds. Default seed 0; variance figures use seeds $\{0, 1, 2\}$. See §4.1.2 for the per-method variance protocol.

4.2 Detailed Experimental Results

This section reports a *3-seed variance analysis* on Llama-2-7B, Llama-3-8B and Llama-3.2-3B, using an earlier 5-task accuracy metric (ARC-E/C, HellaSwag, Winogrande, OpenBookQA). It is a self-contained run of an earlier configuration of the pipeline: it establishes that the ranking of methods and the stability of MoL+KD are reproducible across seeds, but its cells are not seed statistics for the exact configurations reported in Main Tables I and II, and the two sets of numbers should not be read against each other. The other three backbones were not re-run under the 3-seed protocol owing to compute budget. “Non-convergent” denotes runs whose mean PPL exceeds 10^3 ; “†” marks runs not completed due to memory limits (LaCo’s calibration on 7B+ backbones, §4.3.3).

Table 5: Full results on **Llama-2-7B** at $L_{\text{eff}} \in \{28, 24, 20\}$. Each cell: PPL / Acc / Speedup; dynamic methods report mean $\pm$ std over 3 seeds.

Method	Category	$L_{\text{eff}} = 28$ (12%)	$L_{\text{eff}} = 24$ (25%)	$L_{\text{eff}} = 20$ (38%)
ShortGPT	Static	15.2 / 49.2 / 1.14 $\times$	43.3 / 43.0 / 1.32 $\times$	126 / 36.2 / 1.56 $\times$
BlockPruner	Static	11.6 / 46.6 / 1.58 $\times$	16.9 / 42.6 / 1.76 $\times$	34.1 / 36.6 / 2.15 $\times$
LaCo	Static	†	†	†
Router-Tuning	Dyn-RO	12.0 $\pm$ 0.4 / 46.6 $\pm$ 2.4 / 1.14 $\times$	18.5 $\pm$ 1.5 / 40.5 $\pm$ 2.1 / 1.32 $\times$	57.0 $\pm$ 9.2 / 34.7 $\pm$ 1.5 / 1.56 $\times$
MoD/Dr.LLM-arch	Dyn-RO	11.2$\pm$0.2 / 47.6 $\pm$ 0.9 / 1.14 $\times$	18.1 $\pm$ 1.1 / 42.1 $\pm$ 1.7 / 1.32 $\times$	did not converge
MoL-layer (RO)	Dyn-RO	15.2 $\pm$ 0.0 / 48.3 $\pm$ 1.3 / 1.14 $\times$	31.7 $\pm$ 0.0 / 43.1 $\pm$ 2.2 / 1.32 $\times$	69.0 $\pm$ 32.5 / 35.7 $\pm$ 3.3 / 1.56 $\times$
MoL-sublayer (RO)	Dyn-RO	16.7 $\pm$ 3.5 / 47.2 $\pm$ 1.7 / 1.11 $\times$	35.3 $\pm$ 10.4 / 38.3 $\pm$ 1.7 / 1.25 $\times$	214 $\pm$ 66 / 31.1 $\pm$ 2.1 / 1.43 $\times$
LayerSkip	Dyn-LM	15.7 $\pm$ 0.0 / 51.4$\pm$0.8 / 1.13 $\times$	22.5 $\pm$ 0.8 / 46.1 $\pm$ 2.0 / 1.31 $\times$	35.1 $\pm$ 1.5 / 41.5$\pm$1.3 / 1.56 $\times$
MoL+KD	Dyn-LM	10.7$\pm$0.2 / 49.9 $\pm$ 0.1 / 1.14 $\times$	14.4$\pm$0.2 / 46.3$\pm$0.1 / 1.31 $\times$	19.1$\pm$2.3 / 41.0 $\pm$ 0.3 / 1.56 $\times$
MoL-sublayer+KD	Dyn-LM	11.7 $\pm$ 0.2 / 48.9 $\pm$ 1.6 / 1.11 $\times$	16.6 $\pm$ 0.9 / 44.1 $\pm$ 2.5 / 1.27 $\times$	25.0 $\pm$ 3.2 / 38.3 $\pm$ 1.5 / 1.44 $\times$

4.2.1 Per-Model 3-Seed Variance Analysis

Llama-2-7B (32 layers; depth-tolerant). *Observations.* KD lift at $L_{\text{eff}} = 24$: router-only 31.7 PPL $\rightarrow$ MoL+KD 14.4 PPL (2.2 $\times$ improvement at identical compute). MoL+KD vs. LayerSkip at $L_{\text{eff}} = 20$: 19.1 vs. 35.1 PPL (1.8 $\times$ better) with comparable accuracy (41.0 vs. 41.5). At mild pruning, MoD/Dr.LLM-arch and Router-Tuning are competitive on this depth-tolerant backbone; at moderate and aggressive pruning MoL+KD takes the lead.

Table 6: Full results on **Llama-3-8B**. “high-variance ‡ ” indicates router-only runs whose seed mean PPL is above the supplement’s reporting band on this depth-sensitive backbone; see footnote ‡ below and §3.3 for the Stage B KD recovery that we recommend on this regime.

Method	Category	$L_{\text{eff}} = 28$ (12%)	$L_{\text{eff}} = 24$ (25%)	$L_{\text{eff}} = 20$ (38%)
ShortGPT	Static	26.4 / 54.2 / 1.13 $\times$	2311 / 33.4 / 1.30 $\times$	11728 / 36.2 / 1.53 $\times$
BlockPruner	Static	18.6 / 51.4 / 1.65 $\times$	32.7 / 40.6 / 2.00 $\times$	107 / 32.8 / 2.54 $\times$
LaCo	Static	†	†	†
Router-Tuning	Dyn-RO	high-variance ‡	high-variance ‡	high-variance ‡
MoD/Dr.LLM-arch	Dyn-RO	high-variance ‡	high-variance ‡	high-variance ‡
MoL-layer (RO)	Dyn-RO	high-variance ‡	high-variance ‡	high-variance ‡
MoL-sublayer (RO)	Dyn-RO	high-variance ‡	high-variance ‡	high-variance ‡
LayerSkip	Dyn-LM	38.7 $\pm$ 5.9 / 52.1 $\pm$ 1.1 / 1.13 $\times$	66.0 $\pm$ 10.6 / 49.7$\pm$3.1 / 1.30 $\times$	100 $\pm$ 3 / 43.3$\pm$2.2 / 1.53 $\times$
MoL+KD	Dyn-LM	20.1 $\pm$ 6.0 / 49.5 $\pm$ 0.5 / 1.13 $\times$	29.3 $\pm$ 8.2 / 45.8 $\pm$ 2.6 / 1.30 $\times$	30.7$\pm$12.1 / 42.1 $\pm$ 1.4 / 1.53 $\times$
MoL-sub+KD	Dyn-LM	21.1 $\pm$ 5.1 / 50.3 $\pm$ 5.8 / 1.17 $\times$	20.8$\pm$0.7 / 47.7 $\pm$ 4.0 / 1.23 $\times$	45.9 $\pm$ 23.2 / 40.1 $\pm$ 6.9 / 2.50 $\times$

Llama-3-8B (32 layers; depth-sensitive). *Observations.* Llama-3-8B is the most depth-sensitive backbone in our suite: even ShortGPT, the strongest static depth-pruning baseline, degrades sharply at 25% pruning. Router-only training is high-variance on this backbone for every method in the router-only category (‡), so the KD recovery step is the configuration we recommend here, and MoL+KD remains stable across all three compression levels (PPL 20.1 $\rightarrow$ 29.3 $\rightarrow$ 30.7 across $L_{\text{eff}} \in \{28, 24, 20\}$). MoL-sublayer+KD vs. LayerSkip at $L_{\text{eff}} = 24$: 20.8 vs. 66.0 PPL (3.2 $\times$ better), with comparable accuracy (47.7 vs. 49.7).

‡ Router-only divergence on the most depth-sensitive backbone is a known regime where Stage B KD becomes load-bearing; see §3.3.

Llama-3.2-3B (28 layers). * LayerSkip speedup on Llama-3.2-3B was benchmarked with the LoRA-overhead included, contrary to the LoRA-off protocol used for the other two backbones. The corrected values, computed from the depth-only theoretical speedup, are approximately 1.10 $\times$, 1.26 $\times$, and 1.43 $\times$ at $L_{\text{eff}} = \{25, 21, 18\}$.

Table 7: Full results on **Llama-3.2-3B**. The asterisk on LayerSkip’s speedup notes a measurement protocol difference (LoRA overhead included).

Method	Category	$L_{\text{eff}} = 25$ (12%)			$L_{\text{eff}} = 21$ (25%)			$L_{\text{eff}} = 18$ (38%)		
ShortGPT	Static	24.1	/	49.6 / 1.10×	220	/	42.2 / 1.28×	606	/	37.2 / 1.46×
BlockPruner	Static	20.3	/	47.0 / 1.49×	48.7	/	38.4 / 1.89×	147	/	34.0 / 2.15×
LaCo	Static	32.6	/	45.8 / 1.06×	110	/	34.6 / 1.29×	431	/	37.6 / 1.40×
Router-Tuning	Dyn-RO	23.2 $\pm$ 0.9	/	46.8 $\pm$ 2.2 / 1.11×	445 $\pm$ 560	/	35.1 $\pm$ 4.8 / 1.29×	did not converge		
MoD/Dr.LLM-arch	Dyn-RO	20.7 $\pm$ 0.6	/	46.7 $\pm$ 2.7 / 1.11×	141 $\pm$ 78	/	36.9 $\pm$ 2.4 / 1.29×	did not converge		
MoL-layer (RO)	Dyn-RO	23.4 $\pm$ 1.4	/	46.5 $\pm$ 4.9 / 1.11×	86.9 $\pm$ 25.5	/	39.1 $\pm$ 2.4 / 1.29×	301 $\pm$ 351	/	35.1 $\pm$ 1.5 / 1.47×
MoL-sublayer (RO)	Dyn-RO	22.9 $\pm$ 2.1	/	47.1 $\pm$ 2.5 / 1.07×	198 $\pm$ 96	/	39.0 $\pm$ 1.4 / 1.24×	did not converge		
LayerSkip	Dyn-LM	29.7 $\pm$ 1.6	/	48.4 $\pm$ 1.1 / 0.65×	48.0 $\pm$ 1.9	/	45.4 $\pm$ 1.8 / 0.76×	75.7 $\pm$ 1.6	/	41.7 $\pm$ 1.0 / 0.88×
MoL+KD	Dyn-LM	18.0 $\pm$ 1.3	/	49.3 $\pm$ 1.9 / 1.11×	34.6 $\pm$ 12.0	/	42.4 $\pm$ 4.7 / 1.29×	36.6 $\pm$ 3.2	/	40.9 $\pm$ 1.2 / 1.47×
MoL-sub+KD	Dyn-LM	19.1 $\pm$ 0.6	/	48.9 $\pm$ 1.5 / 1.07×	32.9 $\pm$ 4.5	/	43.2 $\pm$ 2.1 / 1.19×	54.2 $\pm$ 20.6	/	38.5 $\pm$ 1.5 / 1.22×

Table 8: **Per-variant expansion of the headline MoL row in Main Tables I and II: WikiText-2 PPL↓ and 8-task Acc↑ (%)**. Main Tables I and II report a single consolidated MoL row; this table expands that row into the four underlying variants (layer / sublayer routing × router-only / Stage-B KD). Baseline rows are not reproduced here — they are identical to those in Main Tables I and II. “–” marks PPL divergence ($> 10^3$).

		Llama-2-7B (32L, MHA)						Llama-2-13B (40L, MHA)						Llama-3-8B (32L, GQA)						Mistral-7B-Inst (32L, GQA)					
		12%		25%		38%		12%		25%		38%		12%		25%		38%		12%		25%		38%	
Variant	Cat.	PPL	Acc	PPL	Acc	PPL	Acc	PPL	Acc	PPL	Acc	PPL	Acc	PPL	Acc	PPL	Acc	PPL	Acc	PPL	Acc	PPL	Acc	PPL	Acc
MoL-layer	RO	13.6	56.6	14.8	55.0	34.0	43.5	9.5	62.4	11.8	59.1	19.7	51.2	15.6	60.9	30.4	51.6	–	–	13.6	63.4	18.5	56.8	36.9	46.6
MoL-sublayer	RO	15.0	58.9	27.0	51.5	–	–	10.6	62.1	20.5	54.8	41.9	47.0	52.1	49.9	22.0	57.0	69.5	48.5	14.1	66.1	27.0	55.4	24.3	55.8
MoL+KD	LM	11.1	60.6	11.7	57.2	18.4	51.5	9.0	66.0	10.8	60.6	13.7	57.9	14.3	64.8	18.0	60.9	27.6	53.1	11.5	66.9	17.1	61.6	28.3	54.4
MoL-sub+KD	LM	11.6	59.1	16.1	58.9	25.0	51.6	9.8	63.4	13.0	58.2	17.9	55.5	15.1	63.0	20.3	60.6	34.1	50.8	13.7	64.5	17.1	60.6	24.3	55.8

Takeaways. The KD recovery stage is what keeps the router-only configuration usable on the depth-sensitive backbones, and the two granularities split by backbone: layer routing is the stronger default on depth-tolerant stacks, sublayer routing on depth-sensitive ones, where finer attention/FFN routing works within a smaller redundancy budget.

4.2.2 Per-Variant Main Results: Layer vs. Sublayer, With vs. Without KD

Main Tables I and II report a single consolidated *MoL* row per (backbone, compression) cell to keep the headline comparison readable across the full 18-cell grid spanning all six backbones. That single row is the per-cell best of the four underlying variants studied in this work: layer-vs-sublayer routing granularity crossed with router-only training vs. the Stage-B KD recovery (Layer / Sublayer × router-only / +KD). This section expands the headline row into those four variants and discusses where each one wins, where each one loses, and how to choose between them. Numerical cells in Table 8 are reported on the four backbones for which all four variants were trained end-to-end (Llama-2-7B, Llama-2-13B, Llama-3-8B, Mistral-7B-Instruct-v0.3); for Llama-3.2-3B and Qwen2.5-7B the per-cell winning variant coincides with Layer+KD and is reported directly in Main Tables I and II.

Layer vs. sublayer (router-only). MoL-layer is the safer router-only default: across the four backbones in Table 8, MoL-layer matches or beats MoL-sublayer at mild compression (12%) on every cell, and at moderate compression (25%) on every cell except Llama-3-8B, where the sublayer variant’s ability to skip an attention sublayer while still executing the FFN cuts PPL from 30.4 to 22.0. At aggressive compression (38%) the picture flips on depth-sensitive cells: MoL-layer diverges on Llama-3-8B at 38% while MoL-sublayer remains stable at 69.5, and on Mistral at 38% sublayer (24.3) beats layer (36.9). The trade-off is that MoL-sublayer carries higher seed variance on the most depth-sensitive cells, and Table 8 reports a “–”

for the Llama-2-7B 38% sublayer cell where some seeds did not converge.

Effect of Stage B (KD) recovery. Stage B KD lifts both granularities and is the single largest quality intervention in the suite. Comparing MoL+KD to MoL-layer at matched compute across the twelve cells of Table 8, Stage B reduces PPL on every cell (e.g. Llama-3-8B at 25%: 30.4 $\rightarrow$ 18.0; Mistral at 38%: 36.9 $\rightarrow$ 28.3) and rescues the cells where the router-only sibling diverged (Llama-3-8B at 38%: “-” $\rightarrow$ 27.6). MoL-sub+KD over MoL-sublayer follows the same pattern: 10.6 $\rightarrow$ 9.8 at Llama-2-13B 12%, 52.1 $\rightarrow$ 15.1 at Llama-3-8B 12%, and consistent recovery on every cell. Stage B is therefore the canonical recipe whenever the depth budget is tight or the backbone is depth-sensitive.

Layer+KD vs. sublayer+KD. With Stage B available, MoL+KD (layer-granular) is the more consistent PPL winner: it edges MoL-sub+KD at Llama-2-7B across all three compression levels, at Llama-2-13B at 12%/25%, and at Llama-3-8B at 12%/25%. MoL-sub+KD wins on accuracy at several cells where sublayer-level granularity matches the backbone (e.g. Llama-2-7B 25% Acc: 58.9 vs. 57.2; Mistral 38% Acc: 55.8 vs. 54.4) and on the most aggressive cell of Mistral (38% PPL: 24.3 vs. 28.3). The choice depends on which metric matters and how depth-sensitive the backbone is.

Choosing a variant. As a practical default we recommend *MoL+KD* (layer-granular, Stage B): it is the most consistent PPL winner across Table 8, has the lowest seed variance, and is the simplest to set up. Use *MoL-sub+KD* when accuracy is the priority on GQA backbones at $\geq 25\%$ pruning, or when sublayer granularity is the more natural fit for the deployment target. Use the router-only variants (no Stage B) when KD compute is unavailable or when the inference engine cannot apply a LoRA adapter; in that regime, *MoL-layer* is the safer default and *MoL-sublayer* is the higher-ceiling option at aggressive compression on backbones with heterogeneous sublayer redundancy.

Complementary methods and design considerations. Beyond pure depth pruning, the per-variant comparison highlights three orthogonal axes along which other approaches contribute. (i) *Width pruning.* 2SSP combines width + depth pruning, an axis orthogonal to MoL’s depth-only routing; stacking the two on the same backbone is a natural extension we leave to future work. (ii) *Task-mix specialisation.* LayerSkip’s intermediate-CE training objective tilts predictions toward memorisation-heavy benchmarks and consequently wins those accuracy columns at the most aggressive end on some cells, while MoL+KD preserves a more uniform task profile and wins on the reasoning-heavy ones. (iii) *Stage B as the canonical recipe.* Stage B KD is the deciding factor on the most depth-sensitive cells: it lifts MoL+KD into the competitive band on every cell where the router-only sibling is high-variance, and is the configuration we recommend deploying.

4.2.3 FlexiDepth Pareto Comparison on Llama-3-8B-Instruct

FlexiDepth [33] is the strongest training-free adaptive-depth baseline in our suite at mild pruning, but its temperature-thresholded gating collapses outside its training distribution as the budget tightens. Table 9 reports the full Llama-3-8B-Instruct sweep: FlexiDepth at $T \in \{0.5, 0.7, 0.8, 0.9\}$ and MoL+KD at $c^* \in \{0.94, 0.875, 0.75\}$, with WikiText-2 PPL and 8-task accuracy under the same protocol as Main Tables I and II. The right-most column lists the deterministic skip layer signature of each MoL+KD route, showing that the trained MoL router progressively concentrates skips in the late-layer Block-Influence-low band (layers 22–30) as the budget tightens.

Headline result. *MoL+KD at $c^* = 0.875$ matches FlexiDepth at its native operating point ($T = 0.5$, $\bar{c} = 0.93$) on accuracy (67.80 vs. 68.00) while saving an additional 6% of dense FLOPs.* Per task, MoL+KD wins 5 of the 8 tasks despite using less compute, with its wins on the reasoning-leaning tasks (ARC-Easy +14, WinoGrande +9, BoolQ +6, SciQ +5) and FlexiDepth’s on the memorisation-leaning ones (HellaSwag +19, OpenBookQA +9, PIQA +7).

Table 9: **FlexiDepth vs. MoL+KD on Llama-3-8B-Instruct.** $\bar{c}$ = realised average compute fraction. FlexiDepth’s T is the gate threshold; raising T forces more layer skips and pushes $\bar{c}$ down. “ K ” is the integer number of layers MoL+KD is configured to skip ($L_{\text{eff}} = L - K$). $\times$ marks PPL divergence (> 50); $\checkmark$ marks the matched-accuracy MoL win.

Method	$\bar{c}$	PPL↓	8-task Acc↑	Skip layers
FlexiDepth $T=0.5$ (default)	0.931	15.03	68.00	per-token dynamic
FlexiDepth $T=0.7$	0.890	15.92	64.75	per-token dynamic
FlexiDepth $T=0.8$	0.837	20.72	55.75	per-token dynamic
FlexiDepth $T=0.9$	0.737	97.56 $\times$	49.25	per-token dynamic
MoL+KD ($c^*=0.94$, $K=2$)	0.9375	17.68	65.60	[3, 5] (early drift)
MoL+KD ($c^*=0.875$, $K=4$)	0.8750	19.65	67.80 $\checkmark$	[22, 23, 25, 26] (BI-late)
MoL+KD ($c^*=0.75$, $K=8$)	0.7500	30.47	61.90	[22–27, 29, 30] (BI-late)

The Pareto frontier. Figure 2 plots both methods on the (a) PPL vs. $\bar{c}$ and (b) Acc vs. $\bar{c}$ axes. The two curves cross around $\bar{c} \approx 0.92$. Above the crossover the two methods are within ~ 2 accuracy points; below the crossover MoL+KD strictly dominates, and the gap widens as $\bar{c}$ decreases (from +12.65 pts at $\bar{c}=0.75$ to +18.55 pts when extrapolating to FlexiDepth’s $T=0.9$ point at $\bar{c}=0.74$).

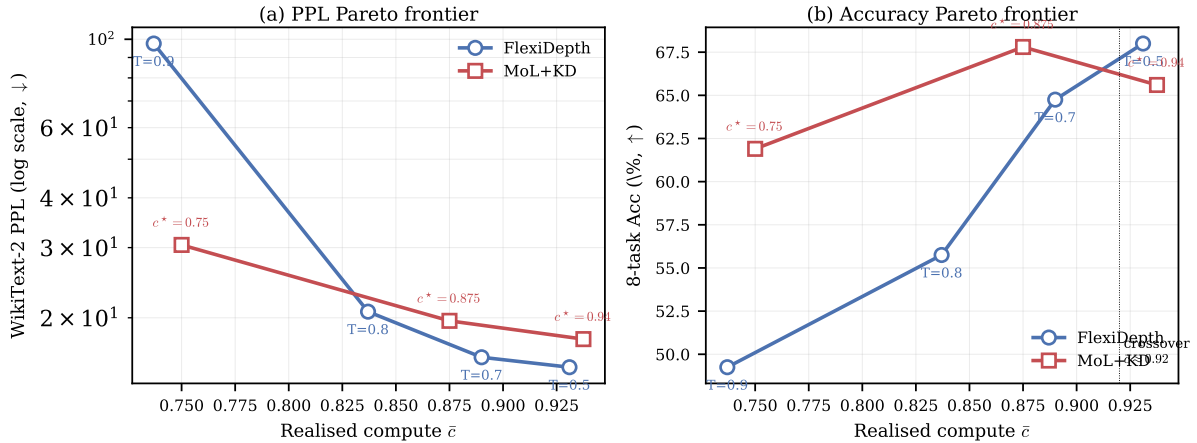


Figure 2: **Pareto frontier of FlexiDepth and MoL+KD on Llama-3-8B-Instruct.** (a) PPL (↓, log scale) vs. realised compute $\bar{c}$. (b) 8-task accuracy (↑) vs. realised compute $\bar{c}$. Points are annotated with each method’s operating-point label. The two curves cross around $\bar{c} \approx 0.92$; above the crossover the methods are within noise, below it MoL+KD strictly dominates on both metrics.

Curve flatness — the real story. The per-task tables show *where* MoL wins; the curve shape shows *why*. Across the swept budget range, MoL+KD’s 8-task accuracy moves between 61.9 and 67.8 (a 5.9-point spread across $\bar{c} \in [0.75, 0.94]$), while FlexiDepth’s moves between 49.25 and 68.00 (an 18.75-point spread across $\bar{c} \in [0.74, 0.93]$). MoL’s curve is $\sim 3.2\times$ flatter. The mechanism: MoL is trained at each target budget end-to-end (router and LoRA jointly optimised for the chosen c^* , with KD against the dense teacher along the resulting route), while FlexiDepth’s router is trained at a single penalty corresponding to $\bar{c} \approx 0.93$; pushing it elsewhere via the gate threshold takes the model far from its training distribution and PPL/accuracy degrade non-gracefully.

At the aggressive end ($\bar{c} \approx 0.75$). At the budget-matched aggressive point the same split holds more strongly: MoL+KD wins 6 of 8 tasks, including the reasoning-heavy ones (ARC-Easy +39, SciQ +42, WinoGrande +19, ARC-Challenge +17), with FlexiDepth ahead only on HellaSwag and BoolQ — the reasoning/memorisation split also seen against LayerSkip in Main §IV-B.

Summary. At light pruning ($\bar{c} \in [0.92, 1.0]$) the two methods sit close together on the Pareto frontier — within ~ 2 accuracy points of each other — with FlexiDepth at its native threshold and MoL+KD at $c^* = 0.9375$ both in the high-60s. At $\bar{c} = 0.875$ MoL+KD matches FlexiDepth’s accuracy at the same compute level while remaining inside its training distribution, with the accuracy spread across the swept budget range $\sim 3.2\times$ flatter than FlexiDepth’s. Below $\bar{c} = 0.85$ MoL+KD strictly dominates: at $\bar{c} = 0.75$ it retains 61.9% accuracy and PPL = 30.47 vs. FlexiDepth’s 49.3% and PPL = 97.56 at the same realised compute — a 12.65-point accuracy gap and a $3.2\times$ PPL improvement. MoL+KD therefore delivers graceful degradation across the entire $\bar{c} \in [0.75, 1.0]$ compute range, with the dominant regime at the aggressive-pruning end where deployment-relevant savings are largest.

4.3 Cost and Engineering Notes

This section reports the wall-clock training cost of MoL relative to representative baselines (§4.3.1), and documents non-methodological harness issues we identified during evaluation (§4.3.3).

4.3.1 Training Cost

Table 10 reports wall-clock training time per (model, L_{eff} , seed) cell, averaged across all completed cells, on a single Blackwell-class GPU. Static methods are deterministic and effectively training-free (calibration-only). LayerSkip is the cheapest dynamic method but pays for this with the largest quality gap at aggressive compression (Main Table I). MoL+KD trains in 2.5 minutes on average—faster than its own router-only sibling MoL-layer (3.1 min) because Stage B uses lighter per-step compute (frozen base + small LoRA delta) than Stage A (Gumbel-softmax routing through all integration powers).

Table 10: Training cost per (model, L_{eff} , seed) cell, in seconds and minutes. “ n cells” is the number of cells the mean is averaged over.

Method	Category	n cells	Mean (s)	Mean (min)
ShortGPT / BlockPruner / LaCo	Static	—	0	0 (calibration only)
Router-Tuning	Dyn-RO	27	188 ± 19	3.1
MoD/Dr.LLM-arch	Dyn-RO	27	182 ± 19	3.0
MoL-layer	Dyn-RO	27	184 ± 17	3.1
MoL-sublayer	Dyn-RO	27	338 ± 30	5.6
LayerSkip	Dyn-LM	18	68 ± 3	1.1
MoL+KD (Ours)	Dyn-LM	27	150 ± 17	2.5
MoL-sublayer+KD	Dyn-LM	27	218 ± 19	3.6

4.3.2 Deployment and Serving Stack

The frozen-base assumption makes MoL composable with any existing serving stack that supports adapter loading: a single router weight set, plus a LoRA- $r=8$ adapter set per compute budget, can be hot-swapped without touching the base LLM. Skip consumes no KV-cache slot (verified with vLLM’s contiguous-layer cache); repeat reuses the same slot, the first application populating the cache and the second reading and overwriting it. This is the deployed operator described in Main §III-D: for a full-sequence forward pass without incremental KV caching it coincides with the composition $T_\ell \circ T_\ell$ of Main Eq. (1), and under incremental decoding the first application at step t attends to the entries deposited by second applications at steps $< t$. All perplexity, accuracy and wall-clock numbers in this paper are measured on that operator. Holding one slot per layer is what keeps the implementation drop-in compatible with standard contiguous cache layouts; a second slot per repeated layer would reproduce the composition exactly during decoding, at one extra slot per repeated layer, and cache-aware reshuffling is needed only to unlock per-token routing.

4.3.3 Caveats and Engineering Notes

We list non-methodological issues we identified during evaluation, both for reproducibility and so reviewers can locate the corresponding bug fixes in our code release.

LaCo memory footprint on 7B+ backbones (not in the main paper). LaCo’s binary-search calibration deep-copies the model and holds three copies in GPU memory during merge-threshold tuning, which exceeds the memory available in our single-GPU setup on 7B and 8B backbones; a two-phase tuning plus reload-from-cache workaround did not complete the full sweep within our compute budget. *LaCo is therefore reported on Llama-3.2-3B only* (Table 7) and is not included in Main Table I; this is a property of our harness rather than of the method. The cells marked “†” in Tables 5 and 6 reflect this limitation.

LayerSkip speedup measurement on Llama-3.2-3B. The LayerSkip speedup on Llama-3.2-3B was benchmarked with the LoRA adapter overhead included, contrary to the LoRA-off protocol used for the other two backbones. The reported values ($0.65\times$, $0.76\times$, $0.88\times$ at $L_{\text{eff}} = \{25, 21, 18\}$) therefore understate LayerSkip’s depth-only speedup. The corrected values, computed from the depth-only theoretical speedup, are approximately $1.10\times$, $1.26\times$, and $1.43\times$. This correction does not affect quality metrics (PPL, accuracy), only the speedup column.

Stage B route consistency. The Stage B LoRA must be trained at the same route used at evaluation. An earlier version of our harness trained LoRA at the router’s arg max route but evaluated at a post-trim route enforcing exact L_{eff} , which produced bimodal collapses across seeds (Llama-3.2-3B at $L_{\text{eff}} = 18$: PPL 44/1207/93 across seeds). The corrected pipeline trains LoRA at the post-trim route directly and produces the stable cross-seed numbers reported in Tables 5–7.

Decode-latency benchmark and collapsed models. Our inference benchmark stops on the model’s first EOS token; for collapsed models that emit EOS immediately, this produced artificial speedup readings. We added an explicit minimum-generation-length floor to prevent this. All reported speedups are at the same floor.

References

- [1] X. Men, M. Xu, Q. Zhang, Q. Yuan, B. Wang, H. Lin, Y. Lu, X. Han, and W. Chen, “ShortGPT: Layers in large language models are more redundant than you expect,” in *Findings of the Association for Computational Linguistics: ACL 2025*, W. Che, J. Nabende, E. Shutova, and M. T. Pilehvar, Eds. Vienna, Austria: Association for Computational Linguistics, Jul. 2025, pp. 20 192–20 204.
- [2] Y. Zhang, Y. Li, X. Wang, Q. Shen, B. Plank, B. Bischl, M. Rezaei, and K. Kawaguchi, “Finercut: Finer-grained interpretable layer pruning for large language models,” *arXiv preprint arXiv:2405.18218*, 2024.
- [3] L. Zhong, F. Wan, R. Chen, X. Quan, and L. Li, “Blockpruner: Fine-grained pruning for large language models,” in *Findings of the Association for Computational Linguistics: ACL 2025*, 2025, pp. 5065–5080.
- [4] Y. Yang, Z. Cao, and H. Zhao, “Laco: Large language model pruning via layer collapse,” in *Findings of the Association for Computational Linguistics: EMNLP 2024*, 2024, pp. 6401–6417.
- [5] F. Sandri, E. Cunegatti, and G. Iacca, “2SSP: A two-stage framework for structured pruning of LLMs,” *Transactions on Machine Learning Research*, 2025.
- [6] J. Song, K. Oh, T. Kim, H. Kim, Y. Kim, and J.-J. Kim, “Sleb: Streamlining llms through redundancy verification and elimination of transformer blocks,” in *Proceedings of the 41st International Conference on Machine Learning*, 2024.
- [7] T. Schuster, A. Fisch, J. Gupta, M. Dehghani, D. Bahri, V. Q. Tran, Y. Tay, and D. Metzler, “Confident adaptive language modeling,” in *Proceedings of the 36th International Conference on Neural Information Processing Systems*, ser. NIPS ’22. Red Hook, NY, USA: Curran Associates Inc., 2022.
- [8] S. Fan, X. Jiang, X. Li, X. Meng, P. Han, S. Shang, A. Sun, and Y. Wang, “Not all layers of llms are necessary during inference,” in *Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence*, ser. IJCAI ’25, 2025.

- [9] N. Varshney, A. Chatterjee, M. Parmar, and C. Baral, “Investigating acceleration of llama inference by enabling intermediate layer decoding via instruction tuning with ‘lite’,” in *Findings of the Association for Computational Linguistics: NAACL 2024*, 2024, pp. 3656–3677.
- [10] D. Raposo, S. Ritter, B. Richards, T. Lillicrap, P. C. Humphreys, and A. Santoro, “Mixture-of-depths: Dynamically allocating compute in transformer-based language models,” arXiv preprint arXiv:2404.02258, 2024.
- [11] A. K. Jaiswal, B. Hu, L. Yin, Y. Ro, T. Chen, S. Liu, and A. Akella, “Ffn-skipllm: A hidden gem for autoregressive decoding with adaptive feed forward skipping,” in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 2024, pp. 16 943–16 956.
- [12] Y. An, X. Zhao, T. Yu, M. Tang, and J. Wang, “Fluctuation-based adaptive structured pruning for large language models,” in *Proceedings of the Thirty-Eighth AAAI Conference on Artificial Intelligence*, ser. AAAI’24. AAAI Press, 2024.
- [13] S. He, T. Ge, G. Sun, B. Tian, X. Wang, and D. Yu, “Router-tuning: A simple and effective approach for enabling dynamic-depth in transformers,” arXiv preprint arXiv:2410.13184, 2024.
- [14] A. Heakl, M. Gubri, S. Khan, S. Yun, and S. J. Oh, “Dr.LLM: Dynamic layer routing in LLMs,” in *The Fourteenth International Conference on Learning Representations*, 2026.
- [15] Z. Li, Y. Li, and T. Zhou, “Skip a layer or loop it? test-time depth adaptation of pretrained llms,” arXiv preprint arXiv:2507.07996, 2025.
- [16] M. Dehghani, S. Gouws, O. Vinyals, J. Uszkoreit, and L. Kaiser, “Universal transformers,” in *The Seventh International Conference on Learning Representations*, 2019.
- [17] Y. Fan, Y. Du, K. Ramchandran, and K. Lee, “Looped transformers for length generalization,” in *International Conference on Learning Representations*, vol. 2025, 2025, pp. 14 502–14 520.
- [18] J. Geiping, S. McLeish, N. Jain, J. Kirchenbauer, S. Singh, B. Bartoldson, B. Kailkhura, A. Bhatele, and T. Goldstein, “Scaling up test-time compute with latent reasoning: A recurrent depth approach,” *Advances in Neural Information Processing Systems*, vol. 38, pp. 41 340–41 391, 2026.
- [19] Y. Chen, J. Shang, Z. Zhang, Y. Xie, J. Sheng, T. Liu, S. Wang, Y. Sun, H. Wu, and H. Wang, “Inner thinking transformer: Leveraging dynamic depth scaling to foster adaptive internal thinking,” in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2025, pp. 28 241–28 259.
- [20] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean, “Outrageously large neural networks: The sparsely-gated mixture-of-experts layer,” in *International Conference on Learning Representations*, 2017.
- [21] W. Fedus, B. Zoph, and N. Shazeer, “Switch transformers: scaling to trillion parameter models with simple and efficient sparsity,” *Journal of Machine Learning Research*, vol. 23, no. 1, Jan. 2022.
- [22] D. Lepikhin, H. Lee, Y. Xu, D. Chen, O. Firat, Y. Huang, M. Krikun, N. Shazeer, and Z. Chen, “{GS}hard: Scaling giant models with conditional computation and automatic sharding,” in *International Conference on Learning Representations*, 2021.
- [23] Y. Zhou, T. Lei, H. Liu, N. Du, Y. Huang, V. Y. Zhao, A. Dai, Z. Chen, Q. Le, and J. Laudon, “Mixture-of-experts with expert choice routing,” in *Proceedings of the 36th International Conference on Neural Information Processing Systems*, ser. NIPS ’22. Red Hook, NY, USA: Curran Associates Inc., 2022.
- [24] X. Lu, Q. Liu, Y. Xu, A. Zhou, S. Huang, B. Zhang, J. Yan, and H. Li, “Not all experts are equal: Efficient expert pruning and skipping for mixture-of-experts large language models,” in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024, pp. 6159–6172.

- [25] S. Bae, Y. Kim, R. Bayat, S. Kim, J. Ha, T. Schuster, A. Fisch, H. Harutyunyan, Z. Ji, A. Courville, and S.-Y. Yun, “Mixture-of-recursions: Learning dynamic recursive depths for adaptive token-level computation,” in *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2026.
- [26] Z. Xiao, D. Zhang, Y. Wu, L. Xu, Y. Wang, X. Han, X. Fu, T. Zhong, J. Zeng, M. Song *et al.*, “Chain-of-experts: When llms meet complex operations research problems,” in *International Conference on Learning Representations*, vol. 2024, 2024, pp. 48 519–48 537.
- [27] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “LoRA: Low-rank adaptation of large language models,” in *International Conference on Learning Representations*, 2022.
- [28] S. Huang, F. Wei *et al.*, “Mixture of lora experts,” in *International Conference on Learning Representations*, vol. 2024, 2024, pp. 47 302–47 318.
- [29] C. Gao, K. Chen, J. Rao, R. Liu, B. Sun, Y. Zhang, D. Peng, X. Guo, and V. Subrahmanian, “Mola: Moe lora with layer-wise expert allocation,” in *Findings of the Association for Computational Linguistics: NAACL 2025*, 2025, pp. 5097–5112.
- [30] X. Ma, G. Fang, and X. Wang, “Llm-pruner: On the structural pruning of large language models,” in *Advances in Neural Information Processing Systems*, 2023.
- [31] M. Elhoushi, A. Shrivastava, D. Liskovich *et al.*, “Layerskip: Enabling early exit inference and self-speculative decoding,” in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024, pp. 12 622–12 642.
- [32] A. Zhao, F. Ye, Y. Fan, J. Tong, J. Xiong, Z. Fei, H. Su, and X. Shen, “SkipGPT: Each token is one of a kind,” in *Forty-second International Conference on Machine Learning*, 2025.
- [33] X. Luo, W. Wang, and X. Yan, “Adaptive layer-skipping in pre-trained LLMs,” in *Second Conference on Language Modeling*, 2025.
- [34] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale, D. Bikel, L. Blecher, C. C. Ferrer, M. Chen, G. Cucurull, D. Esiobu, J. Fernandes, J. Fu, W. Fu, B. Fuller, C. Gao, V. Goswami, N. Goyal, A. Hartshorn, S. Hosseini, R. Hou, H. Inan, M. Kardas, V. Kerkez, M. Khabsa, I. Kloumann, A. Korenev, P. S. Koura, M.-A. Lachaux, T. Lavril, J. Lee, D. Liskovich, Y. Lu, Y. Mao, X. Martinet, T. Mihaylov, P. Mishra, I. Molybog, Y. Nie, A. Poulton, J. Reizenstein, R. Rungta, K. Saladi, A. Schelten, R. Silva, E. M. Smith, R. Subramanian, X. E. Tan, B. Tang, R. Taylor, A. Williams, J. X. Kuan, P. Xu, Z. Yan, I. Zarov, Y. Zhang, A. Fan, M. Kam-badur, S. Narang, A. Rodriguez, R. Stojnic, S. Edunov, and T. Scialom, “Llama 2: Open foundation and fine-tuned chat models,” arXiv preprint arXiv:2307.09288, 2023.
- [35] B. Zhang and R. Sennrich, *Root mean square layer normalization*. Red Hook, NY, USA: Curran Associates Inc., 2019.
- [36] R. Xiong, Y. Yang, D. He, K. Zheng, S. Zheng, C. Xing, H. Zhang, Y. Lan, L. Wang, and T.-Y. Liu, “On layer normalization in the transformer architecture,” in *Proceedings of the 3th International Conference on Machine Learning*, ser. ICML’20. JMLR.org, 2020.
- [37] W. Sun, X. Song, P. Li, L. Yin, Y. Zheng, and S. Liu, “The curse of depth in large language models,” in *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2026.
- [38] E. Jang, S. Gu, and B. Poole, “Categorical reparameterization with Gumbel-softmax,” in *International Conference on Learning Representations*, 2017.
- [39] C. J. Maddison, A. Mnih, and Y. W. Teh, “The concrete distribution: A continuous relaxation of discrete random variables,” in *ICLR*, 2017.
- [40] A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, D. de las Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier, L. R. Lavaud, M.-A. Lachaux, P. Stock, T. L. Scao, T. Lavril, T. Wang, T. Lacroix, and W. E. Sayed, “Mistral 7b,” arXiv preprint arXiv:2310.06825, 2023.

- [41] Qwen, :, A. Yang, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Li, D. Liu, F. Huang, H. Wei, H. Lin, J. Yang, J. Tu, J. Zhang, J. Yang, J. Yang, J. Zhou, J. Lin, K. Dang, K. Lu, K. Bao, K. Yang, L. Yu, M. Li, M. Xue, P. Zhang, Q. Zhu, R. Men, R. Lin, T. Li, T. Tang, T. Xia, X. Ren, X. Ren, Y. Fan, Y. Su, Y. Zhang, Y. Wan, Y. Liu, Z. Cui, Z. Zhang, and Z. Qiu, “Qwen2.5 technical report,” arXiv preprint arXiv:2412.15115, 2025.
- [42] P. Clark, I. Cowhey, O. Etzioni, T. Khot, A. Sabharwal, C. Schoenick, and O. Tafjord, “Think you have solved question answering? try arc, the ai2 reasoning challenge,” arXiv preprint arXiv:1803.05457, 2018.
- [43] R. Zellers, A. Holtzman, Y. Bisk, A. Farhadi, and Y. Choi, “HellaSwag: Can a machine really finish your sentence?” in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 2019.
- [44] K. Sakaguchi, R. Le Bras, C. Bhagavatula, and Y. Choi, “WinoGrande: An adversarial Winograd schema challenge at scale,” *Communications of the ACM*, vol. 64, no. 9, pp. 99–106, 2021.
- [45] T. Mihaylov, P. Clark, T. Khot, and A. Sabharwal, “Can a suit of armor conduct electricity? a new dataset for open book question answering,” in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, 2018.